Big Data Computing — Exam Master Guide

Revised 5 September 2026

Contents

1. Exam priorities and how to use this guide

This guide prepares you for the written exam on **16 September 2026**. It is based on the exercise sheets, five example tests, older exam material, lecture notes and handwritten proofs in this workspace. Explanations and solutions are written for studying; an exam answer should usually be much shorter.

The current written exam lasts **150 minutes** and gives **28 points**. You need both $W \geq 16$ and $W + H \geq 18$, where $H \in [0, 3]$ is your homework score. Therefore, your actual minimum written score is $\max\{16, 18 - H\}$. With 0 homework points you need 18; with 2 or 3 you still need at least 16. Aim for a reliable margin above these thresholds.

The five examples are older **26-point, 120-minute** tests. Their structure is relevant; their duration and maximum score are not the current rules. They contain four short questions and two longer exercises. Four examples contain a long MapReduce design problem; **Example 2 instead contains a clustering proof and a streaming exercise**. Do not assume that every future test must contain exactly the same topic distribution.

What deserves the most practice

Priority	Skills to master	Evidence in the supplied material
Essential	Design 2-3 round MR algorithms; prove local and aggregate space	Long problems in Examples 1, 3, 4, 5; MR and clustering sheets
Essential	FFT, MR-FFT, proxy arguments and diameter bounds	Repeated theory questions; clustering sheet
Essential	Reservoir, Sticky Sampling, Count-Min, Count Sketch, Bloom filters	Theory and long problems throughout the examples
Essential	Exact query definitions, kd-trees, LSH and collision probabilities	Four of the five examples; similarity sheet
High	Spark laziness, timing, partitions and streaming batches	Repeated short questions and homework questions
High	Current fair k-center and frequent-items homeworks	Current rules explicitly allow questions on this year's homeworks
Secondary	Diversity, weighted k-means proofs, hash families, older homework topics	Lecture material and older questions; useful after the essentials

These are study priorities inferred from past material, not predictions of the September paper.

Visual conventions

- [**Theorem**] callouts contain statements worth reproducing accurately.
- [**Exercise TOPIC-N**] headings identify official-sheet exercises and their solutions.
- [**Worked exam exercise/question**] headings identify patterns taken from example papers.
- **Proof** paragraphs stay inside their theorem callout, immediately after the statement.
- **Exercises** sections collect official problems and worked solutions for each chapter.
- Results stated without proof in the sources are explicitly marked; source errors are corrected.
- Figures are selected from the course notes only when they clarify data flow or geometry.

How to study each algorithm

After reading a section, close the guide and write six things:

1. **Problem:** what is the input and what must be returned?
2. **State:** what information is stored?
3. **Procedure:** what happens in each round, update or query?
4. **Guarantee:** exact, approximate, unbiased, or correct with some probability?
5. **Cost:** memory, time, rounds and communication, with assumptions.
6. **Proof idea:** why does the guarantee follow?

Then solve a related exercise without looking at its solution. Reading a proof until it feels familiar is weaker preparation than reconstructing its key steps from a blank page.

Time allocation during the exam

Use approximately 5 minutes to inspect the paper, 50 for short questions, 80 for long exercises and 15 for checking. Adjust to the actual points. If stuck, write the model, an algorithm skeleton and the bounds you can justify, then return later. The examples explicitly warn that excessively long answers are penalized: include the argument that earns the points, not every fact you remember.

2. Mathematical toolkit for proofs

Notation and asymptotic reasoning

We use N for a batch dataset size, n for a stream length or search dataset size, D for dimension, k for the number of centers, L for MR partitions, d for sketch rows and w for sketch width. Context determines whether d instead denotes a distance function.

For fixed-size records, space is counted in words. A word must be large enough to hold an identifier or counter. Thus $O(1)$ words can still mean $O(\log n + \log |U|)$ bits. A D-dimensional point occupies $O(D)$ words unless the exercise explicitly treats it as $O(1)$.

- $f(N) = O(g(N))$: asymptotic upper bound up to a constant.
- $f(N) = \Theta(g(N))$: matching upper and lower bounds.
- $f(N) = o(g(N))$: $f(N)/g(N) \to 0$.

Examples: $\sqrt{N} = o(N)$ and $\sqrt{N}\log N = o(N)$, but $N/100$ is not $o(N)$. Likewise, $\sqrt{Nk} = o(N)$ precisely when $k = o(N)$. Never discard a parameter such as k or K unless the question says it is constant.

Indicator variables and expectation

For an event E, let $I_E = 1$ if E occurs and 0 otherwise. Then

[Theorem] Linearity of expectation

$$\mathbb{E}[I_E] = \Pr(E), \qquad \mathbb{E}\left[\sum_i X_i\right] = \sum_i \mathbb{E}[X_i].$$

Independence is not required.

Proof. Write expectation as a probability-weighted sum over outcomes. Interchanging the two finite sums gives $\mathbb{E}[\sum_i X_i] = \sum_i \mathbb{E}[X_i]$. For an indicator, $\mathbb{E}[I_E] = 1\Pr(E) + 0\Pr(E^c) = \Pr(E)$.

Linearity of expectation does not require independence. This is the main tool for expected partition sizes, sample counts, sketch noise and LSH candidate counts.

An estimator $\hat{\theta}$ is **unbiased** if $\mathbb{E}[\hat{\theta}] = \theta$. This does not mean every run is accurate. Conversely, a biased estimator can still have excellent high-probability error guarantees.

Union bound, Markov and Chebyshev

[Theorem] Union bound

$$\Pr\left(\bigcup_i E_i\right) \le \sum_i \Pr(E_i).$$

Proof. Pointwise, $I_{\cup_i E_i} \le \sum_i I_{E_i}$. Take expectations and use linearity.

Use it when moving from one bad partition or one missed item to the event that **any** of them is bad. No independence is needed.

[Theorem] Markov's inequality

For a nonnegative random variable X and $a > 0$,

$$\Pr(X \ge a) \le \frac{\mathbb{E}[X]}{a}.$$

Proof. Pointwise, $X \ge aI_{X \ge a}$ because $X \ge 0$. Taking expectations and dividing by $a > 0$ proves the bound.

Count-Min uses this on nonnegative collision noise.

[Theorem] Chebyshev's inequality
For a random variable with finite variance,

$$\Pr(|X - \mathbb{E}[X]| \geq a) \leq \frac{\mathrm{Var}(X)}{a^2}.$$

Proof. Apply Markov to the nonnegative variable $(X - \mathbb{E}[X])^2$ with threshold a^2. Its expectation is $\mathrm{Var}(X)$.

Count Sketch uses this because its noise can be positive or negative.

Chernoff bounds and the median trick

[Theorem] Chernoff bounds
Let $X = \sum_i X_i$ for independent Bernoulli variables, with $\mu = \mathbb{E}[X] > 0$. For $\eta > 0$,

$$\Pr(X \geq (1+\eta)\mu) \leq \left(\frac{e^\eta}{(1+\eta)^{1+\eta}}\right)^\mu.$$

For $0 < \eta < 1$,

$$\Pr(X \leq (1-\eta)\mu) \leq e^{-\eta^2\mu/2} \leq 2^{-\eta^2\mu/2}.$$

The course upper-tail form is $\Pr(X \geq a\mu) \leq 2^{-a\mu}$ for $a \geq 6$.
Proof. If $p_i = \Pr(X_i = 1)$, independence and $1 + z \leq e^z$ give

$$\mathbb{E}[e^{tX}] = \prod_i (1 + p_i(e^t - 1)) \leq \exp(\mu(e^t - 1)).$$

For $t > 0$, Markov bounds the upper tail by $\exp(\mu(e^t - 1) - t(1+\eta)\mu)$. Substitute $t = \ln(1+\eta)$. For the lower tail, use $t = \ln(1-\eta) < 0$; now $X \leq (1-\eta)\mu$ implies $e^{tX} \geq e^{t(1-\eta)\mu}$. Markov gives $\exp[-\mu(\eta + (1-\eta)\ln(1-\eta))]$. The bracket is at least $\eta^2/2$: its derivative is $-\ln(1-\eta) \geq \eta$ and both sides start at zero. Finally $e^{-z} \leq 2^{-z}$ for $z \geq 0$. For $a \geq 6$, the upper-tail logarithm per μ is $a - 1 - a\ln a \leq -a\ln 2$, since $\ln a - 1 + 1/a$ is increasing and exceeds $\ln 2$ at 6.

[Theorem] Median amplification
For independent estimates, each outside a fixed valid interval with probability at most $1/16$, the median of an odd number d of estimates fails with probability at most 2^{-d}. Thus $d \geq \log_2(1/\delta)$, rounded up to an odd integer, suffices.
Proof. A bad median requires at least $(d+1)/2$ bad rows. There are at most 2^d subsets of rows; any specified subset of at least $d/2$ rows is entirely bad with probability at most $(1/16)^{d/2}$. Union bound gives $2^d(1/16)^{d/2} = 2^{-d}$. Alternatively, for failure probability at most $1/12$ per row, stochastic domination by $B \sim \mathrm{Bin}(d, 1/12)$ and Chernoff give $\Pr(B \geq d/2) \leq 2^{-d/2}$. Choose $d \geq 2\log_2(1/\delta)$ in that version. Independence between rows is essential.

Three proof patterns to recognize immediately

1. **Packing:** $k+1$ well-separated points fall into k optimal clusters. Two share a cluster, so their distance is at most twice the optimum radius.
2. **Proxy path:** replace a point by a nearby representative and use the triangle inequality to add the errors along the path.
3. **Random noise:** expand an estimator into the true answer plus collision terms; bound their expectation or show that random signs cancel their expectation.

3. MapReduce - model, design patterns and solved exercises

Sources: `1.MapReduce2526`, `BDC_proofs`, `EX-MR2526.pdf`, clustering exercises and the long MR problems in the example tests.

3.1 What MapReduce does

MapReduce processes a dataset of key-value pairs in distributed rounds.

1. **Map:** independently transform each input pair into zero or more intermediate pairs.
2. **Shuffle:** group all intermediate values with the same key.
3. **Reduce:** apply a function to each pair $(key, listOfValues)$ and emit output pairs.

The output of one round becomes the input of the next. The shuffle makes related information meet on one reducer. **The key is the communication decision.**

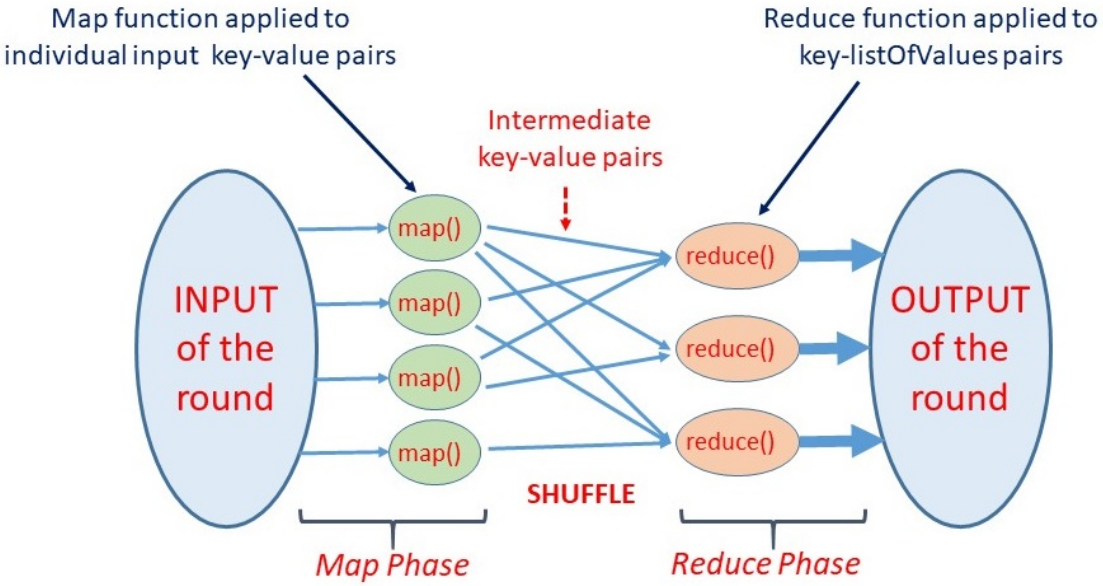


Figure 1 — One MapReduce round: map, shuffle by key, reduce.

The model evaluates:

- R: number of rounds.
- M_L: maximum local space of any individual map or reduce invocation, including its input, output and temporary state under the course model.
- M_A: maximum total space for input, intermediate and output data in a round.

Good targets are $R = O(1)$, $M_L = o(N)$ and $M_A = O(N)$, where achievable for the problem. Time spent inside a reducer still matters, even when an exercise asks only for space.

> **Important — Small output does not imply small local space**
> Sending all N values to key 0 and summing them produces one number, but the reducer receives N values. Under the course model this gives $M_L = \Theta(N)$. Saying "I can sum with a streaming counter" does not satisfy the prescribed reducer-load bound. First split the input and produce bounded-size partial summaries.

3.2 Cluster feasibility and fault tolerance

Each local computation must fit available worker RAM; the total stored data must fit usable distributed storage. For Example 1, the idealized cluster with 10 machines, 8 GB RAM each and 128 GB disk each permits $M_L \leq 8$ GB and $M_A \leq 1280$ GB, ignoring replication and framework overhead. Summing RAM to 80 GB does not permit one 80 GB reducer.

If storage uses replication factor q, physical capacity must cover roughly qM_A, subject to what the exercise counts in M_A. More machines do not fix an oversized individual key.

[Theorem] Mean time between failures (MTBF)
For m components, each failing independently with probability p per time step, and independent steps, the waiting time $T \in \{1, 2, ...\}$ until the first system failure has

$$\mathbb{E}[T] = \frac{1}{1-(1-p)^m} \approx \frac{1}{mp} \quad (mp \ll 1).$$

Proof. A step has no failure with probability $(1-p)^m$. Set $q = 1-(1-p)^m$. Then $\Pr(T > t) = (1-q)^t$, and the tail-sum formula gives $\mathbb{E}[T] = \sum_{t\geq 0}(1-q)^t = 1/q$. Expanding $(1-p)^m$ to first order gives $q \approx mp$. For fixed $p > 0$, the exact expectation tends to 1 as m grows.

Frequent failures motivate replication, fault detection and recomputation.

3.3 Balanced partitioning

Throughout this guide, deterministic "partition by ID" assumes consecutive record IDs. For arbitrary IDs, use independent random assignments and a high-probability load analysis.

[Theorem] Deterministic balanced partitioning
For consecutive record IDs $i \in \{0, ..., N-1\}$, partition i by $i \bmod L$. Every partition has at most $\lceil N/L \rceil$ records.
Proof. Write $N = qL + r$ with $0 \leq r < L$. The q complete blocks of L IDs give each remainder q records; the last r IDs add at most one per remainder. Arbitrary distinct IDs do not suffice: they can all have the same remainder.

Random case. Without suitable IDs, choose a fresh independent uniform partition in $\{0, ..., L-1\}$ for each record. This is not the same as hashing a repeated class label: all occurrences of that label would then move together.

[Theorem] Balanced random partitioning
With $L = \sqrt{N}$ independent uniform assignments, every partition contains $O(\sqrt{N})$ records with probability at least $1 - N^{-5}$, for sufficiently large N.
Proof
For a fixed partition j, write

$$X_j = \sum_{i=1}^{N} I_{ij}, \qquad \Pr(I_{ij} = 1) = 1/L, \qquad \mathbb{E}[X_j] = N/L.$$

For $L = \sqrt{N}$, $\mu = \sqrt{N}$. Chernoff gives

$$\Pr(X_j \geq 6\sqrt{N}) \leq 2^{-6\sqrt{N}}.$$

Union bound over L partitions gives

$$\Pr\left(\max_j X_j \geq 6\sqrt{N}\right) \leq \sqrt{N}\, 2^{-6\sqrt{N}} \leq N^{-5}$$

for sufficiently large N. Thus every partition has $O(\sqrt{N})$ records with high probability. **Expected load of one partition alone is not a bound on the maximum load.**

3.4 The main two-round aggregation pattern

Suppose each record has a logical group g, and each group can be summarized with a constant-size mergeable object: count, sum, maximum, minimum, or a pair of sum and count.

```
Round 1 map:
    (i, record) -> (i mod L, record)
Round 1 reduce on partition b:
    compute one partial summary A[b,g] for each group g present
    emit (g, A[b,g])
Round 2 map:
    identity
Round 2 reduce on group g:
    merge its at most L partial summaries
```

```
emit (g, final_summary)
```

Each group contributes at most one record from each partition, even if it originally occurred N times. Consequently,

$$M_L = O\left(\max\{N/L, L\}\right), \qquad M_A = O(N).$$

Choose $L = \lceil\sqrt{N}\rceil$. Total summaries are at most the number of input records: every emitted summary represents at least one record in its partition.

For an average, merge $(sum, count)$, not averages alone. The average of partition averages is incorrect when partition sizes differ.

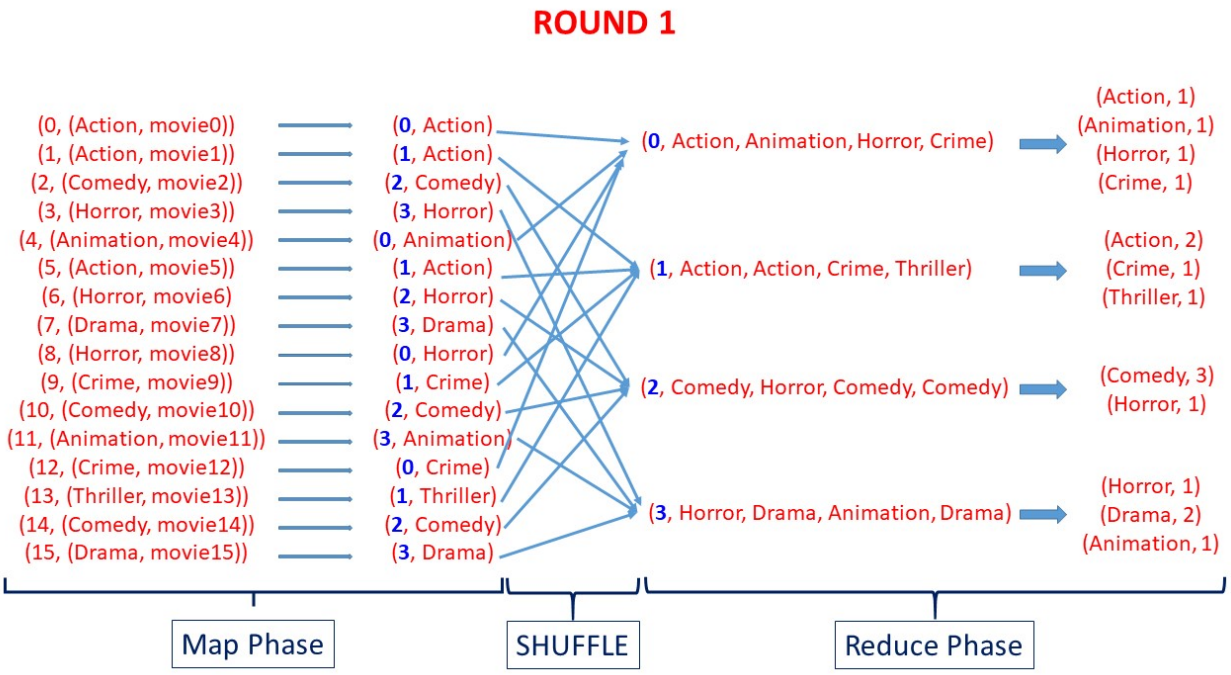


Figure 2 — Round 1: deterministic partitioning produces bounded local class counts.

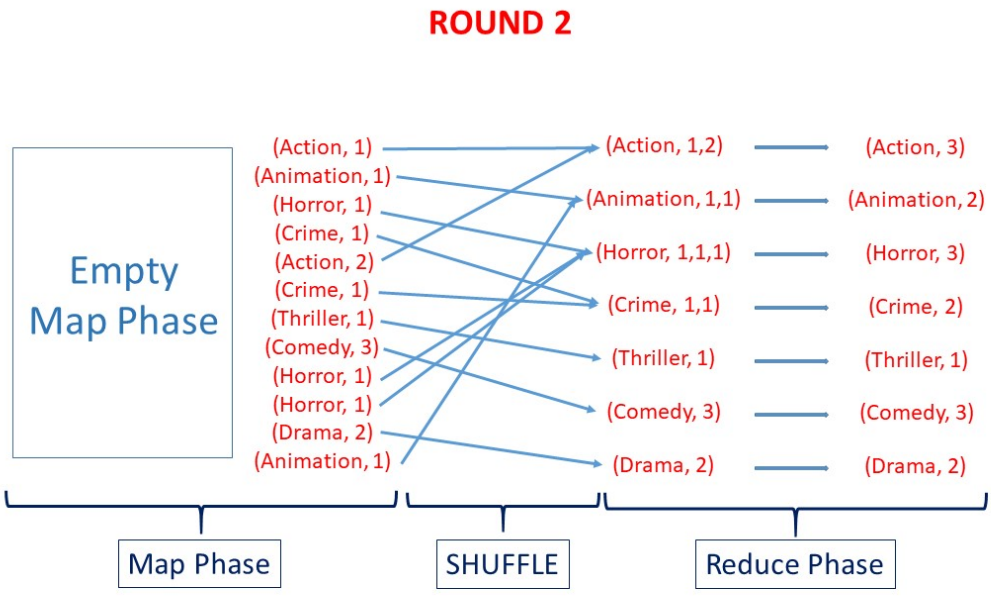


Figure 3 — Round 2: partial counts meet by class and are summed.

3.5 A reusable answer template

For every MR exercise write:

1. Input representation, output representation and global variables.
2. Exact map outputs and reducer outputs for every round.
3. Correctness: which records are summarized, retained or counted exactly once?
4. Local space: largest map input/output, largest reducer input/output, temporary state.
5. Aggregate space: total number **and size** of intermediate records per round.
6. Parameter assumptions and whether a guarantee is deterministic or probabilistic.

If a record contains a vector of k numbers, count $\Theta(k)$ space, not $O(1)$. If each of L partitions emits k summaries, count kL, not merely L.

3.6 Exercises

[Exercise MR-1] Mean and multi-level aggregation

Solution and reasoning.

With $L = \sqrt{N}$, Round 1 emits $(0, s_b)$, where s_b is the sum in partition b. Round 2 returns $(0, \sum_b s_b/N)$. Both reducer inputs have size at most $\sqrt{N}$.

For local space M, group at most M numbers at each level. Successive data sizes are

$$N,\ \lceil N/M \rceil,\ \lceil N/M^2 \rceil, \ldots, 1.$$

With $M \geq 2$, the number of rounds is $\lceil \log_M N \rceil$, local space $O(M)$ and aggregate space $O(N)$. For $M = N^{1/4}$, four rounds suffice. Number partial summaries densely or use the exercise's structured modulo keys so that each later group remains bounded.

General lesson: smaller local memory can be bought with more aggregation levels.

[Exercise MR-2] Exact distinct count without skew

Solution and reasoning.

A naive solution deduplicates by value, then sends all distinct values to one reducer. Its local space is $\Theta(\max\{f_{\max}, D_0\})$, where $f_{\max}$ is the largest multiplicity and D_0 the number of distinct values. Either can be N.

Use four rounds instead, with $L = \lceil \sqrt{N} \rceil$:

1. Partition by ID. Within partition b, keep each distinct x once; emit (x, b).
2. Group by x, receiving at most L partition IDs. Keep one arbitrary owner b and emit (b, x).
3. Group by owner b, count assigned distinct values and emit $(0, c_b)$.
4. Sum the at most L counts.

Correctness: Round 2 leaves exactly one representative of every distinct value. Round 3 counts these representatives once, and Round 4 adds their counts.

The subtle point is Round 3: its owner b was a partition that originally contained x. Therefore owner b cannot receive more distinct values than its original partition size. All reducer inputs are $O(\sqrt{N})$; no round has more than $O(N)$ records.

[Exercise MR-4] Matrix-vector multiplication

Solution and reasoning.

Let A be an $m \times n$ matrix, V an n-vector, and initially $m \leq \sqrt{n}$. Input size is $\Theta(mn)$, but the requested local bound is $o(n)$.

Divide each matrix row and V into segments of $B = \lceil \sqrt{n} \rceil$ positions.

1. Send $A[i, j]$ to key $(i, \lfloor j/B \rfloor)$. Replicate $V[j]$ to the corresponding key for each row i. Tag entries as matrix or vector and retain their coordinate j.
2. Each reducer matches coordinates and computes a partial dot product. Emit it keyed by row i.
3. In the second MR round, sum the $O(\sqrt{n})$ partial products per row.

The first two listed steps are the map and reduce phases of Round 1. There are two rounds in total. Each segment reducer receives $O(\sqrt{n})$ entries. Each vector-entry mapper emits $m \leq \sqrt{n}$ copies. Hence $M_L = O(\sqrt{n})$ and $M_A = O(mn)$.

If m is larger, direct mapper replication may violate local output space. Replicate through multiple stages with fan-out at most $\sqrt{n}$ per invocation, then perform the same dot products. Replication needs $O(\log_{\sqrt{n}} m)$ levels; it remains constant for polynomially bounded m. Count mapper output as well as reducer input.

[Exercise MR-5] At most K records per sensor

Solution and reasoning.

Task: retain all records of a sensor if it has at most K records; otherwise retain exactly K arbitrary records. Preserve occurrences, including equal-valued measurements.

Round 1 partitions by ID. In each partition, keep at most K records per sensor and emit them keyed by sensor. Round 2 receives at most KL retained records per sensor and keeps at most K.

Why enough records survive: if a sensor has at most K records globally, none is discarded. If it has more than K, either some partition retains K, or all its local counts are below K and all its records survive. Either way, at least K candidates reach Round 2.

$$M_L = O\left(\max\{N/L, KL\}\right), \qquad M_A = O(N).$$

For the standard choice $L = \sqrt{N}$:

- constant K: $M_L = O(\sqrt{N})$;
- $K = \log_2 N$: $M_L = O(\sqrt{N}\log N) = o(N)$;
- aggregate space remains $O(N)$ because the algorithm only discards records.

If free to retune L, balance $N/L = KL$, giving $L \asymp \sqrt{N/K}$ and $M_L = O(\sqrt{NK})$. Distinguish changing K in the original algorithm from redesigning the partition count.

[Worked exam exercise] Frequent pairs and biased machines

Solution and reasoning.

Count outcomes identified by the **composite group** (s, o), where s is a machine.

1. Partition records by ID; emit one local count $((s, o), c_{b,s,o})$ per distinct pair.
2. Sum by (s, o). If its total is at least $N/50$, emit (s, o).
3. Group by s and emit (s, null) once.

Rounds 1 and 2 have $O(\sqrt{N})$ load. Round 3 is safe because there are at most 50 frequent pairs **in the whole dataset**: each uses at least $N/50$ of the N records. Thus $R = 3$, $M_L = O(\sqrt{N})$ and $M_A = O(N)$.

The same solution handles frequent products per customer. Read the threshold carefully: $N/50$ is relative to the whole dataset, not to the number of observations for one machine.

[Worked exam exercise] Distinct occupied cells in grid rows

Solution and reasoning.

Task: connections use cells (i, j) of a $t \times t$ grid; return rows using more than $t/2$ distinct cells. Assume $t = O(\sqrt{N})$ and arbitrary multiplicities per cell.

1. Partition records by their distinct connection ID. Locally deduplicate cells and emit $((i, j), 1)$ once per cell per partition.
2. Group by cell (i, j). Receive at most L copies, then emit $(i, 1)$ exactly once.
3. Group by row i. Sum its indicators to obtain t_i; output (i, t_i) if $t_i > t/2$.

No connection multiplicity survives into the final row count. Reducer loads are respectively $O(N/L)$, $O(L)$ and $O(t)$. With $L = \sqrt{N}$,

$$M_L = O(\max\{\sqrt{N}, t\}) = O(\sqrt{N}), \qquad M_A = O(N).$$

Common failed answer: grouping original connections by cell can create an N-record reducer. Another failed answer: summing connections counts traffic volume, not distinct occupied cells.

4. Spark and Word Count

Sources: `2.Spark2526`, `3.WordCountSpark`, `PROJECT_DESCRIPTIONS_EXAM`.

4.1 Architecture and RDDs

Spark executes distributed data-processing operations. The **driver** runs the main application and schedules work. **Executors** run tasks and store partitions. The **cluster manager** allocates resources. An executor is a process, not necessarily an entire physical machine; several tasks may share its resources.

A **Resilient Distributed Dataset (RDD)** is an immutable, partitioned collection that can be processed in parallel. Its **lineage** records how it was derived, allowing lost partitions to be recomputed when the required inputs remain available. A DataFrame adds named columns and a schema; this course's algorithm exercises mainly use RDDs.

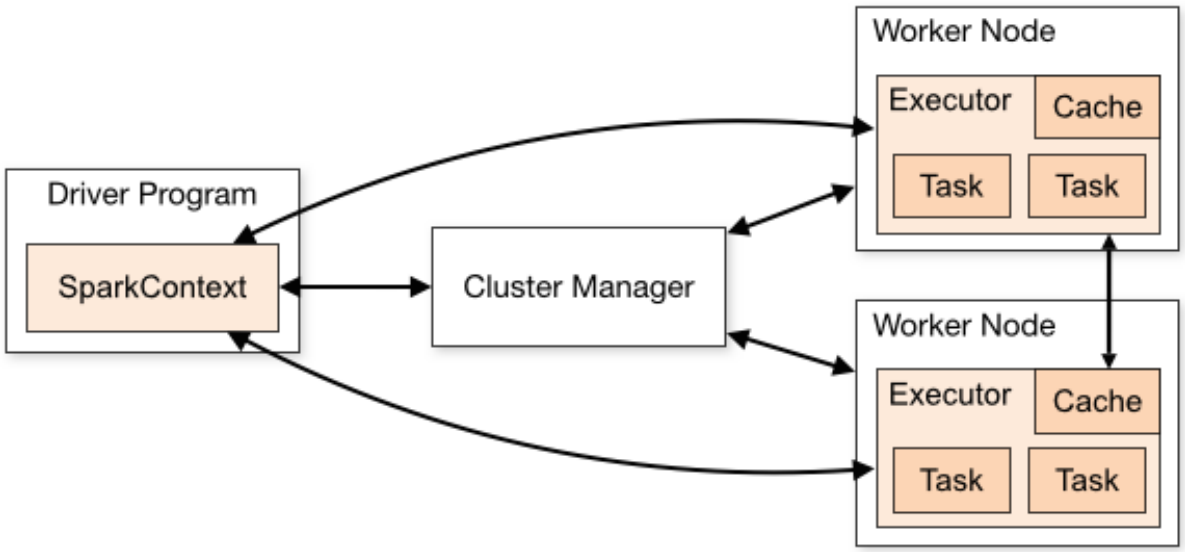


Figure 4 — Spark driver, cluster manager, worker nodes and executors.

4.2 Transformations, actions and timing

Transformations such as `map`, `flatMap`, `filter` and `reduceByKey` describe a new RDD. Actions such as `count`, `collect`, `reduce` or saving output trigger the required computation. **Lazy evaluation** means that timing only the construction of transformations mostly times the creation of a computation plan. Persistence marks data for reuse when materialized; it does not itself perform all the work. These semantics are also documented in the official Spark RDD guide.

```
from time import perf_counter

data = sc.textFile(path).map(parse_line).cache()
data.count()  # Materialize the input before measuring the algorithm.

start = perf_counter()
result = algorithm(data)
answer = result.collect()  # Appropriate only if the final result is small.
elapsed = perf_counter() - start
```

If `algorithm` already executes an action and returns a local object, the action is already inside the timed call. If a reused RDD is not persisted, another action can recompute its lineage. State whether a measurement includes loading, caching and objective evaluation.

4.3 Key operations

Operation	Meaning	Exam relevance
`map(f)`	Apply f once per record	Usually one output per input

Operation	Meaning	Exam relevance
`flatMap(f)`	Emit an iterable of outputs per record	Tokenize documents or replicate records
`mapPartitions(f)`	Apply f to an iterator over one partition	Local counts, local clustering, coreset extraction
`groupByKey()`	Gather values sharing a key	May move and retain many raw values
`reduceByKey(f)`	Combine values per key using an associative merge	Local combining reduces shuffle volume
`collect()`	Return all result records to the driver	Safe only for small results
`count()`	Count records and return a scalar	Triggers computation
`cache()` / `persist()`	Reuse computed partitions under a storage policy	Useful for iterative algorithms

`mapPartitions` is not "map on one record" and not "map on one whole executor". Its function can read many records, maintain local state and emit a smaller summary. Converting its iterator to a list requires the whole partition to fit memory.

A **shuffle** redistributes records, typically by key. Narrow operations such as `map` and `filter` can operate within existing partitions. Grouping by a new key generally requires redistribution. Do not infer the exact number of physical Spark stages merely by counting source-code method calls.

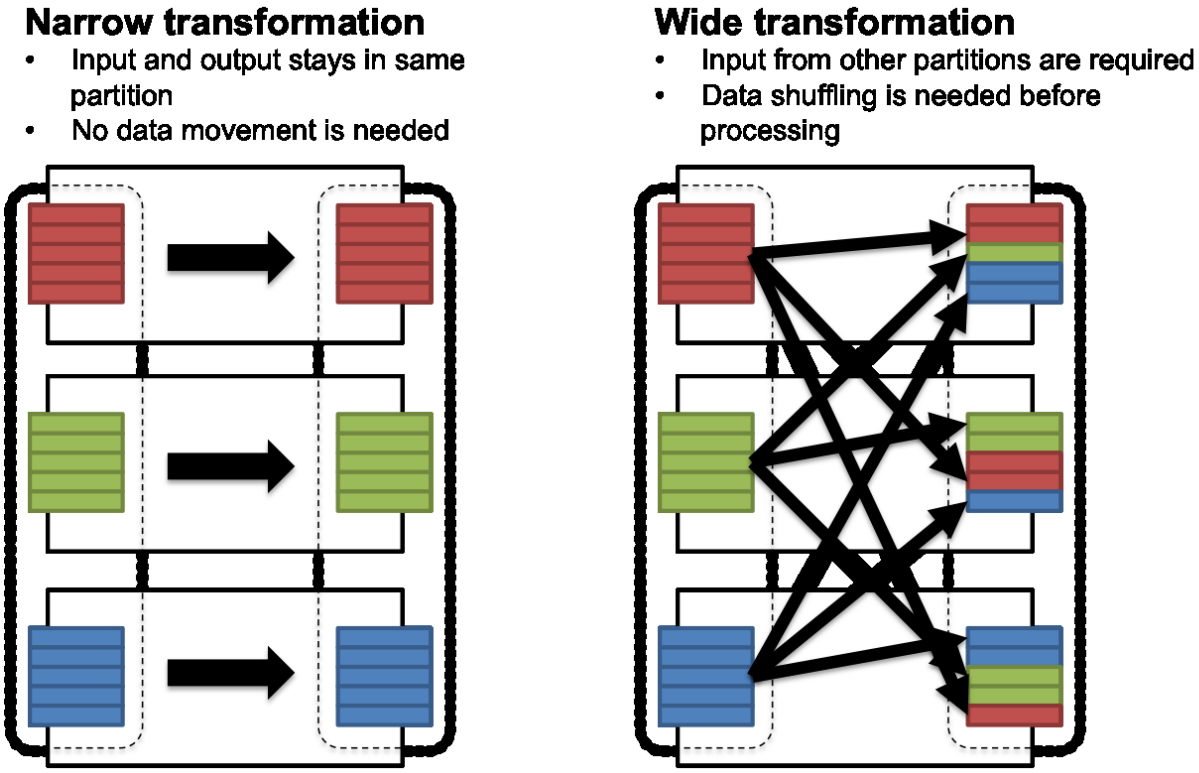


Figure 5 — Narrow dependency versus wide dependency with shuffle.

[Theorem] One-round Word Count local-space bound
Let there be K documents, with total N word occurrences and at most N_{max} words in one document. If each document is first compressed into one count per distinct word, one-round Word Count has $M_L = O(\max\{N_{max}, K\})$ and $M_A = O(N)$ in the course model.
Proof. A mapper reads one document and builds its word-count dictionary using at most $O(N_{max})$ space. For a fixed word, at most one pair comes from each document, so its reducer receives at most K partial counts. Total pairs do not exceed N. Without per-document compression, one word can generate N reducer values. Framework combiners can improve actual execution but do not by themselves prove a skew-independent bound.

4.4 Streaming batches in Spark

In the course's Spark Streaming framework, a **DStream** is represented by a sequence of RDDs, one for each micro-batch. A callback such as `foreachRDD` handles each batch. To run a sequential streaming algorithm, retain its small state across callbacks and feed batch items to its update routine in sequence. A local iterator can avoid collecting a whole batch at once.

This design can distribute ingestion and parsing while still updating the sketch sequentially on the driver. It does not automatically make the streaming update parallel. Partition-local linear sketches can instead be merged when all partitions use identical hash functions and compatible parameters; a reservoir needs a different merge argument. ### 4.5 Exercises

[Exercise MR-3] Word Count

Solution and reasoning.

The task is to return $(word, totalOccurrences)$ over all documents. The basic Spark version is:

```
counts = (documents
          .flatMap(lambda doc: doc.split())
          .map(lambda word: (word, 1))
          .reduceByKey(lambda a, b: a + b))
```

An alternative computes a local dictionary inside each document or partition, emits $(word, localCount)$ pairs, and then merges by word. The arithmetic is identical, but local combining can greatly reduce communication.

For the exercise's two-round randomized MR algorithm:

1. For each document D_i, compute local counts $c_i(w)$. Independently assign each $(w, c_i(w))$ to one of L random partitions.
2. Each partition sums by word and emits $(w, partialCount)$.
3. Round 2 sums the at most L partial counts per word.

Here N is the **total number of word occurrences**, not the number of documents. If $N_{\max}$ is the largest document size and $L = \sqrt{N}$, then

$$M_L = O(N_{\max} + \sqrt{N}) \text{ with high probability}, \qquad M_A = O(N).$$

The document mapper can still be the bottleneck. If a single document has $\Theta(N)$ words, this representation does not give sublinear local space; it must be split if allowed.

5. Clustering and coresets

Sources: `4.Coreset2526-1`, `5.Coreset2526-2`, handwritten proofs pp. 6-14 and `EX-CTCL2526.pdf`.

5.1 Metric spaces and objectives

A **metric** d satisfies nonnegativity, identity of indiscernibles, symmetry and triangle inequality:

$$d(x,y) \geq 0, \quad d(x,y) = 0 \iff x = y, \quad d(x,y) = d(y,x), \quad d(x,z) \leq d(x,y) + d(y,z).$$

For a nonempty center set S, write $d(x,S) = \min_{s \in S} d(x,s)$.

Distance	Formula	Interpretation
L_1, Manhattan	$\sum_j \lvert x_j - y_j \rvert$	Sum of coordinate differences
L_2, Euclidean	$\sqrt{\sum_j (x_j - y_j)^2}$	Geometric distance
L_∞	$\max_j \lvert x_j - y_j \rvert$	Largest coordinate difference
Hamming	Number of differing coordinates	Binary feature mismatch
Jaccard	$1 - \lvert A \cap B \rvert / \lvert A \cup B \rvert$	Set dissimilarity; define two empty sets to have distance 0
Angular	$\arccos(\langle x, y \rangle / (\lVert x \rVert \lVert y \rVert))$	Directional difference for nonzero vectors

Angular distance is a metric on directions or unit vectors, not on arbitrary vectors when distinct positive multiples are treated as different objects. Squared Euclidean distance is not a metric: for points $0, 1, 2$, $4 > 1 + 1$. When working with k-means, use the triangle inequality on distances and only then square with an appropriate inequality.

The three central objectives are:

$$\Phi_{\text{kcenter}}(P,S) = \max_{x \in P} d(x,S),$$

$$\Phi_{\text{kmedian}}(P,S) = \sum_{x \in P} d(x,S), \qquad \Phi_{\text{kmeans}}(P,S) = \sum_{x \in P} d(x,S)^2.$$

k-center controls the worst-served point. **k-median** controls total distance. **k-means** controls squared distance and emphasizes large deviations. All require a center budget $|S| = k$. Always specify whether centers must belong to P (**discrete centers**) or may lie anywhere in the ambient space. This changes some optimal-value comparisons.

A c-approximation for minimization returns a feasible solution with cost at most $c\,\mathrm{OPT}$, where $c \geq 1$. For maximization, it returns value at least OPT/c. Approximation factor and randomized failure probability are different quantities.

5.2 Farthest-First Traversal

Farthest-First Traversal (FFT) selects k centers for metric k-center. It repeatedly chooses the point whose distance from its nearest existing center is largest.

```
FFT(P, k):
    if |P| <= k: return P
    choose an arbitrary c1 in P
    S = {c1}
    for each x in P: nearest[x] = d(x, c1)
    repeat until |S| = k:
        choose c in P \ S maximizing nearest[c]
        add c to S
        for each x in P:
            nearest[x] = min(nearest[x], d(x, c))
    return S
```

Exclude already selected points, especially when coordinates coincide or all remaining distances are zero. Distinct records may have equal coordinates.

Figure 6 — Input point set before Farthest-First Traversal.

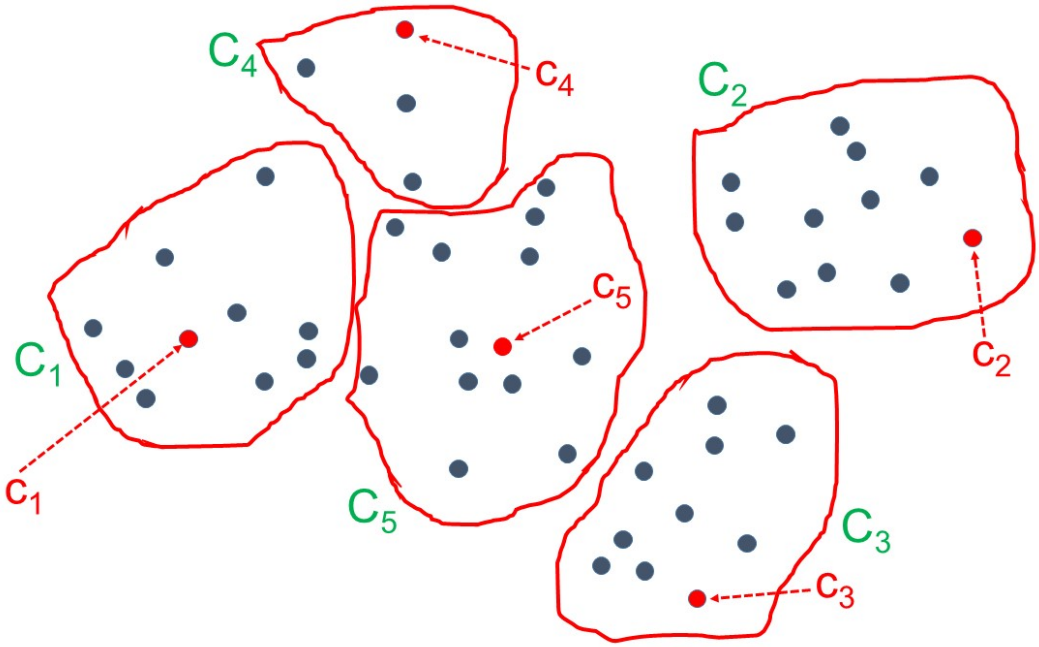


Figure 7 — Centers selected by FFT and induced nearest-center clusters.

5.3 Full proof that FFT is a 2-approximation

[Theorem] FFT approximation guarantee
For metric k-center, Farthest-First Traversal returns k centers of radius $r \leq 2r^*$, where r^* is the optimal radius.
Proof
Let $S = \{c_1, \dots, c_k\}$ be the selected centers, and let q be a point farthest from S. Write $r = d(q, S) = \Phi_{\text{kcenter}}(P, S)$ and $r^* = \text{OPT}$.
If $r = 0$, the conclusion is immediate. Otherwise the $k + 1$ points in $X = \{c_1, \dots, c_k, q\}$ are distinct.
Step 1: every pair in X is at distance at least r. For two selected centers c_i, c_j with $i < j$, let S_{j-1} be the previously selected centers. Then

$$r = d(q, S) \le d(q, S_{j-1}) \le d(c_j, S_{j-1}) \le d(c_j, c_i).$$

The first inequality holds because adding centers cannot increase distance to the set. The second holds because FFT picked a farthest point. The last holds because c_i is one of the previous centers. Also $d(q, c_i) \ge d(q, S) = r$.
Step 2: some pair in X is at distance at most $2r^*$. An optimal solution has k clusters. By the pigeonhole principle, two of the $k+1$ points share an optimal center c^*. Their distance is at most

$$d(a, b) \le d(a, c^*) + d(c^*, b) \le 2r^*.$$

Combining the two steps gives $r \le 2r^*$. Thus FFT is a 2-approximation.
Proof memory cue: add a farthest point; obtain $k+1$ separated points; pigeonhole; triangle inequality.

k-center is sensitive to outliers because even one isolated point can determine the maximum. This is inherent in the objective, not an error in FFT.

5.4 What a coreset is and why composability matters

A **coreset** is a small problem-specific summary with a mathematical guarantee relating solutions on the summary to solutions on the original input. It can be a subset of points, weighted representatives, or another compact structure, depending on the problem.

A **composable coreset construction** permits partitioning P into $P_1, \dots, P_L$, computing summaries T_i independently, then solving on $T = \bigcup_i T_i$ with a controlled loss. An arbitrary union of arbitrary summaries has no such guarantee.

This is effective when local partitions fit workers, the union fits the final solver, summaries are cheap to compute, and their approximation loss is acceptable. It reduces communication and the size of the expensive final computation.

A geometric representative set can be described with a **proxy map** $\tau : P \to T$. If $d(x, \tau(x)) \le a$ for all x, and every representative is within b of the final centers S, then

$$d(x, S) \le d(x, \tau(x)) + d(\tau(x), S) \le a + b.$$

This is the key path: **original point -> representative -> final center**.

[Theorem] FFT subset-cover lemma (local coreset quality)
Let $A \subseteq P$ and let r^* be the optimal metric k-center radius on P. Then $T = \mathrm{FFT}(A, \min\{k, |A|\})$ satisfies $\max_{x \in A} d(x, T) \le 2r^*$.
Proof. If $|A| \le k$, the radius is zero. Otherwise, let R be the final FFT radius. The k selected centers and a farthest point form $k+1$ points with all pairwise distances at least R: each earlier insertion distance is at least the final radius. Two lie in the same one of the k global optimal clusters. Their distance is at most $2r^*$ through its center, so $R \le 2r^*$. This avoids assuming that a discrete optimum decreases on subsets.

5.5 MR-FFT - algorithm, space and full approximation proof

Round 1: partition P into L balanced parts. On each P_i, compute $T_i = \mathrm{FFT}(P_i, k)$, or retain the entire partition if it has fewer than k points. Emit all representatives with a common key for Round 2.

Round 2: collect $T = \bigcup_i T_i$, with $|T| \le kL$, and run $S = \mathrm{FFT}(T, k)$.

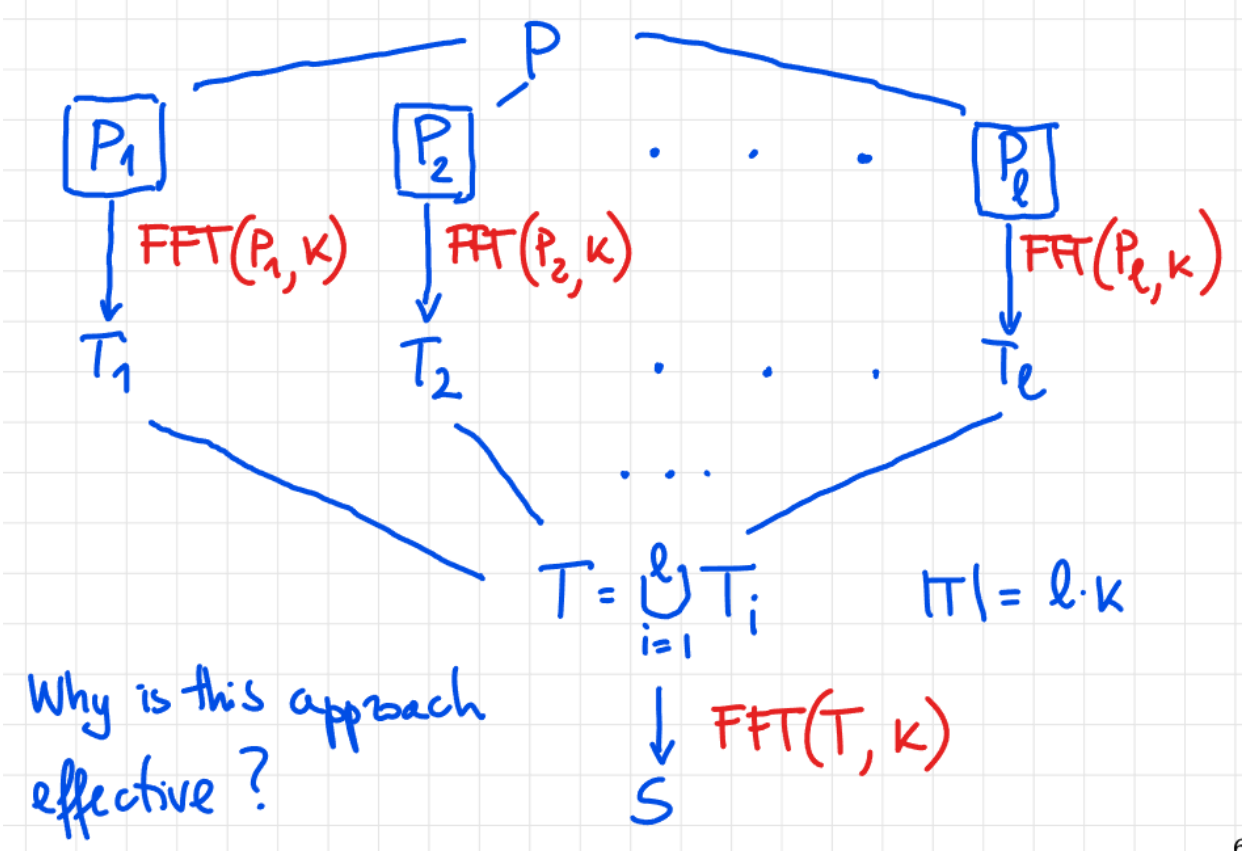


Figure 8 — MR-FFT: local FFT coresets are merged and clustered again.

[Theorem] MR-FFT space complexity
For $1 \le k \le N$, L balanced partitions and linear-space FFT, $M_L = O(\max\{N/L, kL\})$ and $M_A = O(N)$, in point records. Choosing $L = \lceil \sqrt{N/k} \rceil$ gives $M_L = O(\sqrt{Nk})$.
Proof. A first-round reducer stores at most $O(N/L)$ input points; the final reducer receives at most kL representatives. At most N representatives are emitted because local sets never exceed their input sizes. Hence aggregate space stays linear. Balancing $N/L = kL$ minimizes the maximum. Multiply geometric storage by D for D-coordinate points.

Assuming linear-space sequential FFT,

$$M_L = O\left(\max\{N/L, kL\}\right), \qquad M_A = O(N).$$

Balance the two local bottlenecks: $N/L = kL$ implies $L \asymp \sqrt{N/k}$, hence $M_L = O(\sqrt{Nk})$. If points cost $O(D)$ words, include the factor D in stored geometry.

[Theorem] MR-FFT approximation guarantee
With balanced partitions, $L = \lceil \sqrt{N/k} \rceil$ and at most k local representatives per partition, MR-FFT uses $M_L = O(\sqrt{Nk})$, $M_A = O(N)$ and returns a 4-approximation for metric k-center.
Proof
Step 1: local coreset quality
Let r^* be the optimal radius for the full dataset P. On each P_i, the selected local centers plus its farthest point form $k+1$ points of P. The FFT separation argument and the **global** optimal clustering give

$$\max_{x \in P_i} d(x, T_i) \leq 2r^*.$$

Therefore every original point has a proxy in T within $2r^*$.
Step 2: final clustering quality
Apply the same separation argument to FFT on $T \subseteq P$, again comparing against the global optimal clusters. It gives

$$\max_{y \in T} d(y, S) \leq 2r^*.$$

For every $x \in P$, follow its proxy:

$$d(x, S) \leq d(x, \tau(x)) + d(\tau(x), S) \leq 2r^* + 2r^* = 4r^*.$$

Thus **MR-FFT is a 4-approximation**. More partitions decrease local input size but enlarge the final coreset. Oversampling with $h > k$ representatives per partition can improve the proxy radius, but increases the second-round input to hL.

5.6 Why uniform sampling can fail for k-center

Put $N-1$ points in a small region of diameter a, and one point at distance $b \gg a$. For $k = 2$, an optimal solution covers both regions with radius at most a. A uniformly sampled subset of size $m = o(N)$ misses the isolated point with probability $1 - m/N$ when sampling without replacement. If centers must be selected from that sample, both lie in the dense region and the original radius can be about b.

With replacement, the probability of hitting the isolated point is $1 - (1 - 1/N)^m \leq m/N$, so the same failure holds for sublinear sample size.

The approximation ratio can therefore be arbitrarily large. Rare points can be geometrically essential. Sampling is not generally a substitute for a proved coreset construction.

5.7 Diameter - three guarantees you must distinguish

The **diameter** is $\Delta(P) = \max_{x,y \in P} d(x, y)$.

[Theorem] Diameter from one reference point
For nonempty P and $x \in P$, $\Delta(P)/2 \leq \max_{y \in P} d(x, y) \leq \Delta(P)$.
Proof.
One reference point. For any $x \in P$, define $\Delta_x = \max_{y \in P} d(x, y)$. Since these pairs are among all pairs, $\Delta_x \leq \Delta(P)$. For a diameter pair z, w,

$$\Delta(P) = d(z, w) \leq d(z, x) + d(x, w) \leq 2\Delta_x.$$

Thus Δ_x is a 2-approximation to this maximization problem.

[Theorem] Diameter of a representative cover
If $T \subseteq P$ covers P within radius R, then $\Delta(T) \leq \Delta(P) \leq \Delta(T) + 2R$.
Proof.
Direct representative cover. If $T \subseteq P$ and $d(x, T) \leq R$ for every x, choose proxies t_z, t_w for a diameter pair. Then

$$\Delta(T) \leq \Delta(P) \leq d(z, t_z) + d(t_z, t_w) + d(t_w, w) \leq \Delta(T) + 2R.$$

This is the bound requested in Example 5. It is **additive**. If also $R \leq \epsilon \Delta(T)$, it implies the multiplicative bound $\Delta(P) \leq (1 + 2\epsilon)\Delta(T)$.

[Theorem] Diameter with external cluster centers
One input representative per radius-R cluster gives additive diameter loss at most $4R$.
Proof.
Representatives of clusters with external centers. Suppose each original point is within R of an external center c_i, and $t_i \in P$ is an arbitrary point from that center's cluster. Then

$$d(x, t_i) \leq d(x, c_i) + d(c_i, t_i) \leq 2R.$$

Consequently the correct diameter bound is

$$\Delta(T) \leq \Delta(P) \leq \Delta(T) + 4R.$$

Do not write $2R$ in this third situation: replacing the external center introduces another leg at each end of the diameter path.

5.8 k-means++, Lloyd and PAM

k-means++ is randomized initialization. Choose the first center uniformly in the unweighted case. Given selected centers S, choose the next point with probability

$$\Pr(x \text{ next}) = \frac{d(x, S)^2}{\sum_{y \in P} d(y, S)^2}.$$

Points far from existing centers receive more probability.

[Theorem] k-means++ initialization (expectation guarantee)
For squared Euclidean k-means and $k \geq 2$, k-means++ has expected approximation factor $O(\log k)$. For $k = 1$, interpret the bound as $O(1 + \log k)$.
Source scope. The supplied notes state this expectation theorem without its original proof. Exercise CTCL-9 proves the separate Markov/repetition amplification consequence.

The course states this theorem without its full original proof; the probability-boosting exercise CTCL-9 below is the proof technique you need to reconstruct. A theorem for k-means is not automatically the same theorem for k-median or an arbitrary dissimilarity.

Lloyd's algorithm alternates nearest-center assignment and replacing each nonempty cluster center by its mean. Each phase does not increase the squared-Euclidean objective: assignment chooses the cheapest current center, and the mean minimizes the sum of squared distances within a fixed cluster. It can stop at a local optimum and depends on initialization. Handle empty clusters and limit iterations in implementations.

PAM, Partitioning Around Medoids, is a local-search approach with centers chosen among input points. It tries swaps between a selected medoid and an unselected point, retaining improvements. A naive iteration evaluates about $k(N-k)$ candidate swaps; evaluating each swap costs additional work. Merely counting swaps does not make the entire iteration $O(Nk)$ time. It can be expensive on massive inputs.

5.9 Weighted k-means and MR-kmeans

For weights $w(x) \geq 0$, minimize

$$\Phi^w(P, S) = \sum_{x \in P} w(x) d(x, S)^2.$$

Weights record how much original mass a representative stands for. A coreset point representing 1,000 observations should not contribute the same as one representing 1.

Weighted k-means++: choose the first center proportionally to $w(x)$, and later centers with probability

$$\frac{w(x) d(x, S)^2}{\sum_{y \in P} w(y) d(y, S)^2}.$$

If the denominator is zero, the remaining weighted cost is already zero; fill any remaining required centers using a valid convention. Zero-weight points do not attract sampling mass.

[Theorem] Weighted centroid minimizes squared Euclidean cost
For weights $w(x) \geq 0$ with positive total W, the best center for a fixed cluster C is $c = \sum_{x \in C} w(x)x/W$. This is the update in weighted Lloyd's algorithm.
Proof. For any displacement v,

$$\sum_x w(x)\|x - (c+v)\|^2 = \sum_x w(x)\|x - c\|^2 - 2\left\langle \sum_x w(x)(x-c), v \right\rangle + W\|v\|^2.$$

The middle term is zero by the definition of c, so moving away adds $W\|v\|^2 \geq 0$. If $W = 0$, any center has the same zero cost; empty clusters require an explicit convention.

MR-kmeans: locally compute k centers T_i, assign each local point to its nearest representative, and set $w(t) = |\{x : \tau(x) = t\}|$. Gather the weighted union and run a weighted sequential solver. Repeated representative locations must retain or combine their weights, rather than silently discarding multiplicity. The space tradeoff is the same as MR-FFT: $O(\max\{N/L, kL\})$ local, $O(N)$ aggregate.

The course's γ**-coreset** condition is

$$E = \sum_{x \in P} d(x, \tau(x))^2 \leq \gamma\,\mathrm{OPT}.$$

This is a proxy-error definition; it is not identical to every "strong coreset" definition used elsewhere.

[Theorem] MR-kmeans approximation
If proxy error $E \leq \gamma\mathrm{OPT}$ and the weighted solver is an α-approximation on the same feasible center domain, then $\Phi(P, S) \leq [2\gamma + 4\alpha(\gamma + 1)]\mathrm{OPT}$.
Proof.
Assume local and global optima use a common ambient center domain, the local solver is a γ-approximation, and the weighted final solver is an α-approximation, $\alpha \geq 1$. The sum of optimal local costs is at most the global optimum, since global optimal centers are feasible on each part. Thus $E \leq \gamma\mathrm{OPT}$.
Using $(a+b)^2 \leq 2a^2 + 2b^2$ with global optimal centers S^*,

$$\Phi^w(T, S^*) = \sum_x d(\tau(x), S^*)^2 \leq 2E + 2\mathrm{OPT}.$$

For the final centers S,

$$\Phi(P, S) \leq 2E + 2\Phi^w(T, S) \leq 2E + 2\alpha\Phi^w(T, S^*) \leq [2\gamma + 4\alpha(\gamma + 1)]\mathrm{OPT}.$$

This proves the stated $O((1+\gamma)\alpha)$ approximation. If each partition restricts centers to its own input subset, global centers may be infeasible locally. State the domain assumption or account for an additional representative-conversion factor; do not use the local-optimum comparison without justification.
For input-restricted local centers, convert every nonempty optimal local cluster to an input representative. In Euclidean squared distance, choosing the point nearest its mean costs at most twice the unrestricted optimum: the mean identity gives $\sum_x \|x - t\|^2 = \sum_x \|x - \bar{x}\|^2 + |C|\|t - \bar{x}\|^2 \leq 2\sum_x \|x - \bar{x}\|^2$. For a general metric with squared cost, choose the point nearest the ambient center and use $(a+b)^2 \leq 2a^2 + 2b^2$ to get factor 4. Thus local γ-approximation implies proxy error at most $2\gamma\mathrm{OPT}$ or $4\gamma\mathrm{OPT}$, respectively. Substitute that factor in the theorem; the asymptotic guarantee remains $O((1+\gamma)\alpha)$.

5.10 Diversity maximization and distinct proxies

For **max-sum diversity**, select $S \subseteq P$, $|S| = k$, maximizing

$$\mathrm{div}(S) = \sum_{\{x,y\} \subseteq S} d(x, y).$$

The sum is over unordered pairs, hence $\binom{k}{2}$ terms. A $(1+\epsilon)$-coreset T satisfies $\mathrm{OPT}_{div}(T, k) \geq \mathrm{OPT}_{div}(P, k)/(1+\epsilon)$. Running a c-approximation on T then gives a $c(1+\epsilon)$-approximation on P. The course mentions a $(2-2/k)$ sequential approximation as the final solver; its internal proof is not developed in the supplied lectures.

Construct $h \geq k$ FFT clusters of radius R. Keep $\min\{k, |C_i|\}$ points in each cluster, including its center. The coreset has at most hk points.

[Theorem] Diversity coreset via injective proxies
For $2 \le k \le |P|$, retain up to k representatives per radius-R cluster. The resulting T satisfies $\mathrm{OPT}_{div}(T,k) \ge \mathrm{OPT}_{div}(P,k) - 4R\binom{k}{2}$.
Proof.
Why retain multiple representatives? An optimal diverse set can select several points from one cluster. Mapping all of them to the same center would lose the required cardinality k. Keeping up to k representatives permits an **injective** proxy map from the optimal set: there are enough distinct proxies in every cluster.
Each proxy is at distance at most $2R$ through the cluster center. Therefore, for any optimal pair x, y,

$$d(\tau(x), \tau(y)) \ge d(x,y) - 4R.$$

Summing gives

$$\mathrm{div}(\tau(S^*)) \ge \mathrm{OPT}_{div} - 4R\binom{k}{2}.$$

To connect this to k-center, run k steps of FFT on P. Its selected centers have pairwise distances at least its final radius, which is at least the optimal k-center radius r^*. Their diversity is therefore at least $\binom{k}{2}r^*$, implying

$$r^* \le \frac{\mathrm{OPT}_{div}}{\binom{k}{2}}.$$

If $R \le r^*/8$, the loss is at most half the optimal diversity, so T is a 2-coreset. More generally, $4R\binom{k}{2} \le \eta \mathrm{OPT}_{div}$ yields a $1/(1-\eta)$-coreset for $0 < \eta < 1$. Notice that $1/(1-\eta)$ is not exactly $1+\eta$.

To obtain exactly a $(1+\epsilon)$-coreset, it suffices that $R \le \epsilon r^*/[4(1+\epsilon)]$. Then loss is at most $\epsilon \mathrm{OPT}_{div}/(1+\epsilon)$. A bound $(1-\eta)\mathrm{OPT}_{div}$ corresponds to factor $1+\eta/(1-\eta)$, not $1+\eta$. For $k=1$, diversity is identically zero.

5.11 Optional background - repeated coreset compression

Merge-and-reduce maintains summaries at increasing size levels: merge compatible summaries when a level fills, then compress the union to a new summary at the next level. It can turn a mergeable coreset construction into a streaming summary with logarithmically many active levels.

Approximation errors accumulate. If each compression multiplies error by at most $1+\eta$ and a point passes through h levels, the factor can be $(1+\eta)^h$. To target $1+\epsilon$, choose per-level accuracy accordingly, for example $\eta \le \ln(1+\epsilon)/h$. This is supplementary background; the supplied central clustering algorithms use the two-round construction above.

5.12 Exercises

[Exercise CTCL-1] Prove that L_1 is a metric
Solution and reasoning.

The sheet says "L_1 (Euclidean)"; L_1 is Manhattan. Its nonnegativity and symmetry follow coordinatewise; the sum is zero exactly when every coordinate agrees. Finally,

$$|x_j - z_j| \le |x_j - y_j| + |y_j - z_j|$$

for every coordinate, and summing proves the triangle inequality.

[Exercise CTCL-2] Assign each point to its nearest global center
Solution and reasoning.

Each mapper holds the array of k centers, scans it, and emits $(ID_x, (x, \arg\min_j d(x, c_j)))$. No grouping is needed to label points; use an identity reduce if a round is required. Local space is $O(k)$, aggregate data space $O(N)$ for constant-size points. If you instead group all points by cluster ID, you introduce a potentially huge reducer that the assignment task does not need.

[Exercise CTCL-3] Analyze FFT running time
Solution and reasoning.

Each newly selected center requires one scan of N points and one distance computation per point. Cached nearest distances therefore give $O(Nk)$ time for constant-time distances, or $O(NkD)$ in D dimensions. Auxiliary memory is $O(N+k)$. Recomputing distances to every previously selected center from scratch would waste a factor of k.

[Exercise CTCL-4] An accurate coreset

Solution and reasoning.

Suppose $T \subseteq P$ and $d(x,T) \le \epsilon r^*$ for all $x \in P$. Run FFT on T. The separation argument gives $d(y,S) \le 2r^*$ for all $y \in T$, so

$$\Phi_{\text{kcenter}}(P,S) \le (2+\epsilon)r^*.$$

[Exercise CTCL-5] Optimal radius of a subset

Solution and reasoning.

With centers constrained to the dataset being clustered, it is not always true that $\text{OPT}(T,k) \le \text{OPT}(P,k)$: the best original centers may have been removed.

From every nonempty intersection $T \cap C_i^*$, choose a representative t_i as a center. Any other point of that intersection is at distance at most $2r^*$ from t_i through the old center. Add arbitrary centers if fewer than k were chosen. Therefore

$$\text{OPT}(T,k) \le 2\text{OPT}(P,k).$$

The factor is tight: $P = \{-1,0,1\}$, $T = \{-1,1\}$ and $k = 1$ give discrete optimal radii 1 and 2. For general k, add $k-1$ sufficiently distant isolated points to both sets. If arbitrary ambient centers remain available, the original optimum is feasible on T and the stronger monotonicity bound does hold.

[Exercise CTCL-6] One point per cluster of external centers

Solution and reasoning.

Partition by ID into $L = \sqrt{N}$ groups. Each partition determines the closest center for its points and emits one representative per nonempty cluster, keyed by cluster index. Round 2 keeps one point per cluster. Each reducer sees at most L representatives. Global center storage costs $O(k)$, so $M_L = O(\sqrt{N}+k) = O(\sqrt{N})$ when $k = O(\sqrt{N})$; $M_A = O(N)$. The diameter approximation is exactly the external-center bound $\Delta(T) + 4R$ above.

[Exercise CTCL-7] Exact diameter with quadratic aggregate space

Solution and reasoning.

One explicit construction follows the official solution. Assume $\sqrt{N}$ is integral for notation:

1. Replicate each (i, x_i) into $\sqrt{N}$ records $((i,a), x_i)$.
2. Replicate each into $\sqrt{N}$ records $((i, a\sqrt{N}+b), x_i)$, creating one copy for every ordered pair (i,j).
3. Map each to key $(\min\{i,j\}, \max\{i,j\})$. Each reducer gets x_i, x_j and emits distance $d(x_i,x_j)$ with integer key $iN + j$. Handle a diagonal key by emitting zero.
4. Aggregate maxima using successive key moduli $N^{3/2}$, N and $\sqrt{N}$, then a final common key. Each level groups at most $\sqrt{N}$ values.

There are seven rounds as written; two replication/distance stages can be combined. Every map invocation emits at most $\sqrt{N}$ records, every reducer receives at most that many, and at most $O(N^2)$ records exist in any round. Thus $R = O(1)$, $M_L = O(\sqrt{N})$ and $M_A = O(N^2)$. Output one maximum per reducer, not all locally computed distances in one giant list.

[Exercise CTCL-8] Approximate diameter in two rounds

Solution and reasoning.

With global reference point x, partition by ID, compute each partition's maximum distance from x, then take the maximum of the L partial maxima. Local space $O(\sqrt{N})$, aggregate space $O(N)$ and quality $\Delta_x \le \Delta(P) \le 2\Delta_x$.

[Exercise CTCL-9] Boost k-means++ success

Solution and reasoning.

If one run has cost at most αOPT with probability at least $1/2$, run t independent instances and return the lowest-cost solution. It fails only if all runs fail: probability at most 2^{-t}. Choose $t = \lceil \log_2 N \rceil$ for failure at most $1/N$. This is a **best-of-runs** argument, not the median trick. If only an expectation bound is given, first use Markov to obtain constant success, allowing a constant-factor change in α.

[Exercise CTCL-10] Unit-square grid coreset

Solution and reasoning.

A cell of side $1/c$ has Euclidean diameter $\sqrt{2}/c$. Retain one point from every nonempty cell. Each point is within $\sqrt{2}/c$ of its cell representative, so

$$\Delta(T) \le \Delta(P) \le \Delta(T) + \frac{2\sqrt{2}}{c}.$$

An additive error alone is not a uniform relative error bound when the true diameter can be arbitrarily small.

[Exercise CTCL-11] Farthest point of each cluster

Solution and reasoning.

Use the two-round group-summary pattern. Inside each partition, retain the point of each cluster with maximum distance to its center. Round 2 takes the maximum over that cluster's at most L candidates. Compare pairs $(distance, pointID)$ for consistent tie-breaking. Space is $O(\sqrt{N})$ local and $O(N)$ aggregate for constant-size records, since each input already includes its center.

[Exercise CTCL-12] Monochromatic clusters

Solution and reasoning.

Use a two-bit mask as summary: red gives `01`, blue gives `10`, and merge is bitwise OR. Locally merge masks by cluster, then merge again in Round 2. Output the corresponding color for `01` or `10`, and -1 for `11`. The same $O(\sqrt{N})$ and $O(N)$ bounds hold. Standard statements assume clusters are nonempty; an empty cluster needs a specified output convention.

[Exercise CTCL-13] Weighted closest neighboring center

Solution and reasoning.

For each cluster i and candidate center $j \neq i$, locally accumulate

$$A_{b,i,j} = \sum_{x \in P_b \cap C_i} w_x d(x, c_j)^2.$$

Emit one vector of k totals per cluster per partition, keyed by i. Round 2 adds vectors and returns the minimizing $j \neq i$. With constant k, each vector is constant-size, giving $M_L = O(\sqrt{N})$ and $M_A = O(N)$. Randomly partition records if consecutive IDs are unavailable. If k varies, vector sizes and repeated distance calculations must be included: the constant-k analysis no longer applies unchanged.

[Worked exam exercise] Furthest center by average distance

Solution and reasoning.

Replace weighted squared distance by $d(x, c_j)$, and maximize over j, including $j = i$ if allowed. For a fixed nonempty cluster, all candidates have the same denominator $|C_i|$, so maximizing the sum already maximizes the average. If averages themselves are requested, also merge counts.

Older 08/09/2023 question - nearest dataset point to each global query. Partition P; in each partition find its nearest point to each $q \in Q$, then merge candidates by q. Each partition emits k records; Round 2 receives L per query. Thus

$$M_L = O(\max\{N/L, k, L\}), \qquad M_A = O(N + kL).$$

For $L = \sqrt{N}$ and $k \le \sqrt{N}$, the desired bounds hold. If $k > \sqrt{N}$ and L is unchanged, aggregate space becomes $O(N + k\sqrt{N})$. For $k = o(N)$, retuning $L \asymp N/k$ gives local space $O(k + N/k) = o(N)$ and aggregate space $O(N)$. For $k = \Theta(N)$, the globally stored query array itself prevents sublinear local space in this design.

[Worked example] Quick FFT trace

[Example] Quick FFT trace
Let $P = \{0, 2, 3, 10\}$, $k = 2$, first center 0. Distances are $0, 2, 3, 10$, so the second center is 10. Final nearest distances are $0, 2, 3, 0$, giving radius 3. Centers $\{2, 10\}$ give radius 2. FFT need not be optimal; its theorem bounds how far it can be from optimal on any metric input.

6. Streaming algorithms and solved exercises

Sources: `6. Streaming2526-1`, `7.Streaming2526-2`, handwritten proofs pp. 15-23, `EX-STR2526.pdf` and example tests.

6.1 Streaming model and evaluation criteria

A stream is a sequence $\Sigma = x_1, x_2, ...$ whose items arrive one at a time. A streaming algorithm updates compact working state and must answer a query about the prefix seen so far. Data may be unbounded or too large to store.

Evaluate four resources:

- **working memory**, ideally sublinear or polylogarithmic in stream length or universe size;
- **number of sequential passes**, ideally one;
- **update time per item**, ideally constant or logarithmic;
- **query time**, ideally independent of the stream length.

Accuracy must also be specified: exact, additive or relative error; deterministic or probabilistic; per query or simultaneously for every possible query. A structure with fixed size $d \times w$ can still require $O(dw \log n)$ bits because counters grow with n.

The main tools are **sampling**, which keeps selected observations, and **sketching**, which stores randomized linear summaries. Sampling can return original items. A sketch normally answers queries but cannot enumerate all keys unless candidate keys are maintained separately.

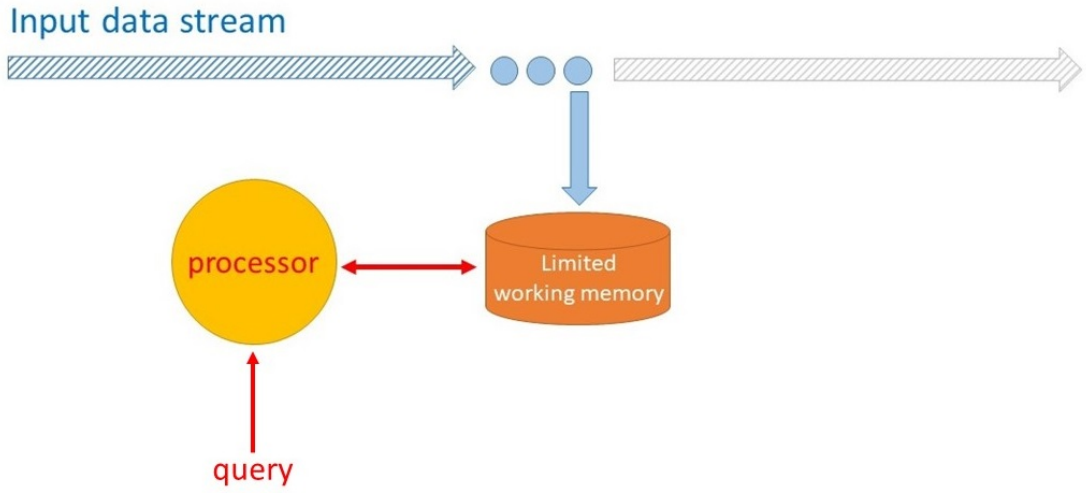


Figure 9 — Stream processor with limited working memory.

6.2 Boyer-Moore majority vote

An item is a majority if its frequency exceeds $n/2$. Boyer-Moore maintains candidate `cand` and integer `count`:

```
count = 0
for x in stream:
    if count == 0:
        cand = x
        count = 1
    else if x == cand:
        count += 1
    else:
        count -= 1
return cand
```

It uses one pass, $O(1)$ words and $O(1)$ update/query time. If a majority exists, returned candidate is that majority. If none exists, the returned candidate can be arbitrary. Exact verification needs a second pass or an independently maintained exact frequency for the final candidate, which cannot generally be recovered retroactively in one pass.

[Theorem] Boyer-Moore majority-vote correctness
If an item occurs more than $n/2$ times, Boyer-Moore returns that item.
Proof (cancellation invariant). After processing t items, the prefix can be partitioned into `count` occurrences of `cand` and $(t - count)/2$ unequal pairs.
Proof by induction:

- If old count is zero, new item becomes the single unpaired candidate occurrence.
- If new item equals candidate, add it to unpaired candidate occurrences.
- Otherwise, pair the new item with one previously unpaired candidate occurrence; count falls by one.

If majority a existed but final candidate differed, every occurrence of a would lie inside an unequal pair. Each such pair contains at most one a, so $f_a \leq n/2$, contradiction.

6.3 Reservoir Sampling

An m**-sample** of t distinct records is a subset S_t of size m such that each record is included with probability m/t. Reservoir Sampling does not require the final stream length:

```
Store first m records.
For record x_t with t > m:
    with probability m/t:
        replace one uniformly random reservoir position by x_t
```

Use indexed reservoir storage so selecting and replacing a random position costs $O(1)$. The algorithm samples **record occurrences**. Equal values can appear several times in the reservoir.

[Theorem] Reservoir Sampling uniformity
After processing $t \geq m$ records, each record belongs to the size-m reservoir with probability exactly m/t.
Proof by induction
At $t = m$, inclusion probability is 1. Assume an old record x_i has probability $m/(t-1)$ before processing x_t. It is evicted with conditional probability

$$\Pr(x_t \text{ inserted}) \Pr(x_i \text{ selected} \mid x_i \in S_{t-1}) = \frac{m}{t} \cdot \frac{1}{m} = \frac{1}{t}.$$

Therefore

$$\Pr(x_i \in S_t) = \frac{m}{t-1}\left(1 - \frac{1}{t}\right) = \frac{m}{t}.$$

New x_t is included with probability m/t, completing induction.

6.4 Frequent Items and Sticky Sampling

For threshold $\phi \in (0, 1)$, an item is frequent when $f_x \geq \phi n$. There can be at most $1/\phi$ frequent items. The ϵ**-Approximate Frequent Items** problem requires an output F satisfying:

1. every item with $f_x \geq \phi n$ belongs to F;
2. no item with $f_x < (\phi - \epsilon)n$ belongs to F.

Items in the grey zone $[(\phi - \epsilon)n, \phi n)$ may be returned or omitted.

For known n, Sticky Sampling sets

$$r = \left\lceil \frac{\ln(1/(\delta\phi))}{\epsilon} \right\rceil, \qquad p = \min\{r/n, 1\}.$$

It maintains a hash table of sampled items and lower-bound counters:

```
for each arrival x:
    if x is already stored: counter[x] += 1
    else with probability p: counter[x] = 1
return every stored x with counter[x] >= (phi - epsilon)n
```

[Theorem] Sticky Sampling guarantee
With probability at least $1-\delta$, every item with frequency at least ϕn is returned and no item below $(\phi-\epsilon)n$ is returned. Expected memory is $O(r)$.
Proof of correctness and expected space
If $r > n$, probability must be capped at 1. Ceiling placement matters: use $r = \lceil \ln(1/(\delta\phi))/\epsilon \rceil$, not $\lceil \ln(1/(\delta\phi)) \rceil/\epsilon$.
The stored counter ignores occurrences before first successful sampling, so it never exceeds true frequency. Therefore no item below $(\phi-\epsilon)n$ is returned, deterministically.
Assume $n \geq 1$, $0 < \epsilon < \phi \leq 1$ and $0 < \delta < 1$. If $p = 1$, all counts are exact. Otherwise, for a frequent item a, let $m_a = \lceil \epsilon n \rceil$. Sampling any of its first m_a occurrences leaves at least $f_a - (m_a - 1) > f_a - \epsilon n \geq (\phi - \epsilon)n$ counted occurrences. Consequently,

$$\Pr(a \text{ missed}) \leq (1-p)^{m_a} \leq (1-r/n)^{\epsilon n} \leq e^{-\epsilon r} \leq \delta\phi.$$

Union bound over at most $1/\phi$ frequent items gives probability at most δ that any frequent item is missed. Hence the two AFI conditions hold simultaneously with probability at least $1-\delta$.
Each stream occurrence creates a new entry with probability at most r/n. With indicator I_t for entry creation,

$$\mathbb{E}[|S|] = \sum_t \mathbb{E}[I_t] \leq np = \min\{n, r\}.$$

Repeated occurrences of an already stored item create no new entry, which only improves the bound. Expected working memory and final scan time are $O(r)$; hash-table update time is $O(1)$ expected. This memory guarantee is in expectation, while correctness probability is at least $1-\delta$: do not merge those statements.
For unknown stream length, the lecture sketches geometrically growing batches and decreasing sampling rates, with periodic downsampling of stored entries. Memorize the idea and say that full details are omitted from the slides; do not invent a complete theorem unless asked.

6.5 Frequency moments and probabilistic counting

For universe U, let f_u be frequency of item u. The k-th frequency moment is

$$F_k = \sum_{u \in U} f_u^k,$$

with $0^0 = 0$. Important cases are F_0, number of distinct values; $F_1 = n$ for an insertion-only stream; and F_2, which measures concentration. The course's Gini expression is $1 - F_2/n^2$: it is 0 when one value occupies the entire stream and approaches 1 as mass is spread across many equally frequent values.

Probabilistic Counting chooses a uniform b-bit hash $h : U \to \{0, \ldots, 2^b - 1\}$, where $2^b \geq |U|$. Let $\mathrm{tr}(z)$ be number of trailing zero bits. Maintain

$$R = \max_{x \text{ seen}} \mathrm{tr}(h(x)),$$

and return $\widetilde{F}_0 = 2^R$. Repetitions of the same item have the same hash and do not change R, which is why the estimator depends on distinct items.

For a uniform bit string,

$$\Pr(\mathrm{tr}(h(x)) \geq j) = 2^{-j}.$$

With F_0 distinct items, the expected number reaching level j is $F_0/2^j$, so the largest occupied level lies around $\log_2 F_0$.

[Theorem] Flajolet-Martin / Probabilistic Counting: constant-factor accuracy
Use uniform b-bit hashes, $2^b \geq |U|$, with $\mathrm{tr}(0) = b$. For a nonempty stream, let $m = F_0$ and $\widetilde{F}_0 = 2^R$. For $c > 2$, the upper-tail probability is at most $1/c$. Under fully independent hashes, the lower-tail probability is also at most $1/c$, so success in $[m/c, cm]$ is at least $1 - 2/c$. With only pairwise independent hashes, the rounding-safe lower-tail bound proved here is $2/c$, giving success at least $1 - 3/c$. Store an empty-stream flag and return 0 if empty.
Proof: upper tail. Set $j = \lfloor \log_2(cm) \rfloor + 1$. If $j > b$, the event is impossible. Otherwise $2^R > cm$ implies some distinct item has at least j trailing zeros. Union bound gives $\Pr(2^R > cm) \leq m2^{-j} < 1/c$. Only uniform marginal hash values are needed.
Proof: lower tail. If $m/c \leq 1$, underestimation is impossible. Otherwise set $j = \lceil \log_2(m/c) \rceil$ and let Y count distinct items with at least j trailing zeros. The bad event is $Y = 0$, with $\mu = \mathbb{E}[Y] = m2^{-j} > c/2$. Under pairwise independence, $\mathrm{Var}(Y) \leq \mu$, so Chebyshev gives $\Pr(Y = 0) \leq 1/\mu < 2/c$. Under full independence,

$$\Pr(Y = 0) = (1 - 2^{-j})^m \leq e^{-m2^{-j}} < e^{-c/2} \leq 1/c.$$

The last inequality holds for $c > 2$. This product formula requires full independence; it does not follow from pairwise independence. The course's $1/c$ lower-tail statement suppresses integer-threshold details; the distinction above makes the assumptions explicit.

The register R uses $O(\log \log |U|)$ bits; the course quotes $O(\log |U|)$ bits including compact hash parameters. That compact implementation uses limited-independence hashing and the corresponding bound above. A fully random hash oracle is an idealized assumption; its full lookup table is not included in the register-space bound.

6.6 Count-Min Sketch

Count-Min maintains a nonnegative $d \times w$ array and independent row hashes $h_j : U \to \{0, \ldots, w - 1\}$.

```
update(x, delta = 1):
    for each row j:
        C[j,h_j(x)] += delta

query(u):
    return min_j C[j,h_j(u)]
```

[Theorem] Count-Min point-query guarantee
With $w = \lceil 2/\epsilon \rceil$ and $d = \lceil \log_2(1/\delta) \rceil$, a fixed insertion-only query satisfies $f_u \leq \tilde{f}_u \leq f_u + \epsilon n$ with probability at least $1 - \delta$.
Proof
For insertion-only nonnegative updates,

$$C[j, h_j(u)] = f_u + \sum_{a \neq u : h_j(a) = h_j(u)} f_a \geq f_u.$$

Therefore Count-Min never underestimates, but each row and the minimum are generally biased upward. The minimum selects the least contaminated row.
Fix a row. Collision error E_j is nonnegative and, with pairwise-uniform hashing,

$$\mathbb{E}[E_j] \leq \frac{n - f_u}{w} \leq \frac{n}{w}.$$

Set $w = \lceil 2/\epsilon \rceil$. Then $\mathbb{E}[E_j] \leq \epsilon n/2$, so Markov gives $\Pr(E_j > \epsilon n) \leq 1/2$. Since final estimate is bad only when all independent rows are bad,

$$\Pr(\tilde{f}_u - f_u > \epsilon n) \leq 2^{-d}.$$

With $d = \lceil \log_2(1/\delta) \rceil$, for a fixed query u,

$$f_u \leq \tilde{f}_u \leq f_u + \epsilon n$$

with probability at least $1 - \delta$. To claim this simultaneously for every item in a set of q queries, use a union bound and build for failure probability δ/q.
Memory is $O(dw)$ counters or $O(dw \log n)$ bits, update/query time $O(d)$. For detecting frequent items at threshold ϕn, Count-Min has no false negatives in an insertion-only stream, but collisions can cause false

positives. A separate set of observed candidates is needed because a matrix alone cannot enumerate universe keys.

6.7 Count Sketch and signed updates

Count Sketch uses row hashes h_j and independent sign hashes $g_j : U \to \{-1, +1\}$:

```
update(x, delta):
    for each row j:
        C[j,h_j(x)] += delta*g_j(x)

row_query(u,j):
    return g_j(u)*C[j,h_j(u)]

query(u):
    return median_j row_query(u,j)
```

[Theorem] Count-Sketch point-query guarantee
Each row estimator is unbiased and has variance at most F_2/w. With $w = \Theta(1/\epsilon^2)$ and $d = \Theta(\log(1/\delta))$, the median has additive error at most $\epsilon\sqrt{F_2}$ with probability at least $1 - \delta$.
Proof
For a fixed row,

$$\tilde{f}_{u,j} = f_u + \sum_{a \neq u} f_a g_j(a) g_j(u) I[h_j(a) = h_j(u)].$$

Every collision term has expectation zero because the sign product is equally likely to be $+1$ or -1. Hence $\mathbb{E}[\tilde{f}_{u,j}] = f_u$. Use pairwise independent uniform signs, independent of bucket hashes, and universal bucket hashes. Expanding the squared error eliminates cross terms because $g_j(u)^2 = 1$ and $\mathbb{E}[g_j(a)g_j(b)] = 0$ for $a \neq b$. Therefore

$$\mathrm{Var}(\tilde{f}_{u,j}) \leq \frac{1}{w} \sum_{a \neq u} f_a^2 \leq \frac{F_2}{w}.$$

Choose $w = \lceil 16/\epsilon^2 \rceil$. Chebyshev makes one row fail with probability at most $1/16$, so it satisfies $|\tilde{f}_{u,j} - f_u| \leq \epsilon\sqrt{F_2}$ with constant probability. Taking the median of $d = \Theta(\log(1/\delta))$ independent rows makes failure at most δ.
The unbiasedness theorem applies to each **row estimator**. The median of unbiased random variables is not automatically unbiased; claim its high-probability accuracy, not unbiasedness. Count Sketch handles signed turnstile updates naturally, while Count-Min's no-underestimate property relies on nonnegative updates.

Examples 2 and 4 - weighted or net frequencies. For records (u, z), where update weight z can be $+1, -1, 1/2$ or $1/3$, update every row by $z g_j(u)$. Then

$$\tilde{f}'_{u,j} = g_j(u) C[j, h_j(u)]$$

is unbiased for $f'_u = \sum_{records\ of\ u} z$. Use median across rows for robustness.

- Net sales: $z = +1$ for purchase and -1 for return.
- Colored frequency: $z = 1/2$ for red and $1/3$ for blue.

If all stream records have product p, no other **key** collides with it, so every row is exact for its net sales. Likewise, if no other item shares u's cell in one row, that row is exact for the weighted frequency; the same key's red and blue updates are desired signal, not collision noise.

6.8 Estimating the second moment with Count Sketch

[Theorem] Unbiased second-moment row estimator
For one Count-Sketch row, $\widetilde{F}_{2,j} = \sum_b C[j,b]^2$ is unbiased. With 4-wise independent signs, $w = \lceil 32/\epsilon^2 \rceil$ and an odd $d \geq \log_2(1/\delta)$ independent rows, its median satisfies $|\widetilde{F}_2 - F_2| \leq \epsilon F_2$ with probability at least $1 - \delta$.
Proof
For row j, define

$$\widetilde{F}_{2,j} = \sum_{b=0}^{w-1} C[j,b]^2.$$

Expanding all squared buckets gives

$$\widetilde{F}_{2,j} = \sum_u f_u^2 + 2\sum_{u<v} f_u f_v g_j(u) g_j(v) I[h_j(u) = h_j(v)].$$

Every cross term has expectation zero, so $\mathbb{E}[\widetilde{F}_{2,j}] = F_2$. Use one unordered pair $u < v$ with coefficient 2, or use ordered pairs $u \neq v$ without coefficient 2; mixing both conventions double-counts noise.
With 4-wise independent uniform signs, independent of universal bucket hashes, distinct unordered-pair cross terms in the squared noise have zero expectation: at least one sign occurs just once. Thus

$$\mathrm{Var}(\widetilde{F}_{2,j}) \leq \frac{4}{w}\sum_{u<v} f_u^2 f_v^2 = \frac{2}{w}\left(F_2^2 - \sum_u f_u^4\right) \leq \frac{2F_2^2}{w}.$$

Choose $w = \lceil 32/\epsilon^2 \rceil$. Chebyshev gives $\Pr(|\widetilde{F}_{2,j} - F_2| > \epsilon F_2) \leq 1/16$. An odd $d \geq \log_2(1/\delta)$ and the median theorem give failure at most δ. Pairwise signs suffice for unbiasedness; 4-wise signs justify this variance calculation.

Important — Source correction: second-moment accuracy
Slide 47 and derivative notes print $\epsilon\sqrt{F_2}$ as error for estimating F_2. For the stated sketch width, the proved bound is ϵF_2. The printed claim does not follow from the variance and is not a valid general guarantee. Explain the discrepancy when discussing that slide; do not present its formula as a proved theorem. Scaling every frequency by t scales the estimator and its error by t^2, whereas the printed error allowance scales only by t.

6.9 Bloom filters

A Bloom filter represents a set S of m elements with an n-bit array A, initialized to zero, and k independent hashes $h_j : U \to \{0, \ldots, n-1\}$.

```
insert(x): set A[h_j(x)] = 1 for every j
query(x): return PRESENT iff every A[h_j(x)] equals 1
```

[Theorem] Bloom-filter guarantees
Inserted elements never cause false negatives. Under ideal uniform hashing, a nonmember's false-positive probability is approximately $(1 - e^{-km/n})^k$, minimized near $k = (n/m)\ln 2$.
Derivation
There are no false negatives: insertion sets exactly the positions later checked, and ordinary updates never clear bits. A nonmember can be a false positive because other items may have set all queried positions.
After km independent uniform bit selections,

$$\Pr(A[i] = 0) = \left(1 - \frac{1}{n}\right)^{km} \approx e^{-km/n}.$$

Approximating queried bit events as independent gives

$$p_{FP} \approx (1 - e^{-km/n})^k.$$

For fixed m, n, this is minimized near

$$k^* = \frac{n}{m}\ln 2,$$

To check the optimum, put $a = m/n$ and differentiate $\log p_{FP}(k) = k \ln(1 - e^{-ak})$:

$$\frac{d}{dk} \log p_{FP}(k) = \ln(1 - e^{-ak}) + \frac{ak}{e^{ak} - 1}.$$

Its zero is $e^{-ak} = 1/2$, giving $k = \ln 2/a$; the derivative is negative before this point and positive after it. Compare the two neighboring positive integers when rounding. At the optimum, about half the bits remain zero and $p_{FP} \approx (0.6185)^{n/m}$. These are approximations under ideal hashing, not exact independence statements for overlapping query positions.
Bloom filters support OR for union when dimensions and hashes agree. Bitwise AND is not a safe exact representation of intersection: a member of $S_1 \cap S_2$ survives, so no false negative for actual intersection members under matching hashes, but collision-set semantics are not equivalent to inserting only the intersection, and interpretation needs care.

6.10 Universal hashing and fingerprinting

A family $\mathcal{H}$ from universe U to $[m]$ is **2-universal** if for distinct x, y,

$$\Pr_{h \sim \mathcal{H}}(h(x) = h(y)) \leq 1/m.$$

It is strongly 2-universal if every ordered output pair occurs with probability exactly $1/m^2$. Strong universality implies uniform marginals and independence; ordinary 2-universality only controls collisions.

For prime $p > |U|$,

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m,$$

with $a \in \{1, \ldots, p-1\}$ and $b \in \{0, \ldots, p-1\}$, yields the practical family used in the course. Reduction modulo m after modulo p is not perfectly uniform unless m divides p; the collision theorem still gives the required bound.

[Theorem] Carter-Wegman practical 2-universal hash family
Let p be prime, encode U in $\{0, \ldots, p-1\}$, and choose $a \in \{1, \ldots, p-1\}$ and $b \in \{0, \ldots, p-1\}$ uniformly and independently. Then $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$ has collision probability at most $1/m$ for distinct keys.
Proof. For distinct x, y and any ordered distinct residues r, s, equations $ax + b = r$, $ay + b = s$ modulo p have exactly one solution: $a = (r - s)(x - y)^{-1} \bmod p \neq 0$ and $b = r - ax \bmod p$. Thus their residues are uniform over $p(p-1)$ ordered distinct pairs. Fixing the first residue, at most $\lceil p/m \rceil - 1 = \lfloor (p-1)/m \rfloor$ other residues share its remainder modulo m. Conditional collision probability is therefore at most $1/m$; averaging preserves this bound. Nonzero a makes distinct pre-reduction outputs unequal, so this is not the same as strongly 2-independent output hashing. Two parameters require $O(\log p)$ bits.

More generally, **k-universal collision control** bounds the probability that k distinct keys all collide by $1/m^{k-1}$. **Strong k-universality** instead requires their outputs to be jointly uniform, with probability $1/m^k$ for every prescribed output tuple. For a Mersenne prime $p = 2^q - 1$, writing $x = 2^q h + l$ gives $x \equiv h + l \pmod{p}$. This proves the bit-folding reduction used to implement modular hashing; repeat folding and normalize to $[0, p-1]$ as needed.

A **fingerprint** stores a short hash of an object and compares fingerprints before doing expensive exact work. Equal objects have equal deterministic fingerprints; unequal objects can collide with controlled probability. It is one-sided Monte Carlo equality testing, not a frequency estimator.

For approximate membership, storing m fingerprints of b bits gives no false negatives and false-positive probability at most $m/2^b$ by union bound. Under fully independent fingerprints it is $1 - (1 - 2^{-b})^m$. Setting $b \geq \lceil \log_2(m/\delta) \rceil$ suffices for error at most δ, using $O(m \log(m/\delta))$ bits. Bloom filters share bit positions and achieve $O(m \log(1/\delta))$ bits under their approximate analysis.

6.11 Choosing the right streaming method

Need	Method	Central guarantee	Main limitation
Majority candidate	Boyer-Moore	Finds majority if one exists	Needs verification to reject false candidate

Need	Method	Central guarantee	Main limitation
Uniform sample	Reservoir	Every record has inclusion probability m/t	Does not directly ensure all frequent values
Approximate frequent-item set	Sticky Sampling	AFI correct with probability $1-\delta$	Expected dictionary size; known-n version
Distinct-count order of magnitude	Probabilistic Counting	Constant-factor probability bound	One copy is noisy
Nonnegative point frequency	Count-Min	One-sided additive error	Biased upward; cannot enumerate keys alone
Signed/weighted point frequency	Count Sketch	Unbiased rows; two-sided error	Wider than Count-Min for stated error
Second moment	Squared Count-Sketch buckets	Unbiased rows	Use corrected relative error ϵF_2
Approximate membership	Bloom filter	No false negatives; controlled FP	Cannot list set; ordinary deletion unsafe

6.12 Exercises

[Exercise STR-1] Boyer-Moore invariant

Solution and reasoning.

After processing t items, the prefix can be partitioned into `count` occurrences of `cand` and $(t - count)/2$ unequal pairs.

Proof by induction:

- If old count is zero, new item becomes the single unpaired candidate occurrence.
- If new item equals candidate, add it to unpaired candidate occurrences.
- Otherwise, pair the new item with one previously unpaired candidate occurrence; count falls by one.

If majority a existed but final candidate differed, every occurrence of a would lie inside an unequal pair. Each such pair contains at most one a, so $f_a \le n/2$, contradiction.

[Exercise STR-2] Missing number

Solution and reasoning.

For n distinct stream values from $\{1, \dots, n+1\}$, maintain $S = \sum_t x_t$. Return

$$\frac{(n+1)(n+2)}{2} - S.$$

This uses $O(1)$ machine words, assuming a word can store sums of order n^2. XOR over $1, \dots, n+1$ and all stream values is an overflow-safe alternative. If n is unknown, count arrivals and evaluate the formula at query time.

[Exercise STR-3] Expected frequent occurrences

Solution and reasoning.

If value a appears $f_a \ge \phi n$ times, let X be its number of occurrences in the final reservoir. Every occurrence has inclusion probability m/n, so

$$\mathbb{E}[X] = f_a \frac{m}{n} \ge \phi m \ge 1$$

when $m \ge 1/\phi$. Expected count at least one does **not** prove that the sample contains a with probability 1, nor even with a specified high probability.

[Exercise STR-4] Merge two equal-size reservoirs

Solution and reasoning.

If two length-n streams are disjoint at record level and S_1, S_2 are independent m-samples, choose a uniform m-sample S from their $2m$ stored records. For record x in the first stream,

$$\Pr(x \in S) = \frac{m}{n} \cdot \frac{m}{2m} = \frac{m}{2n},$$

and symmetrically for the second stream. This proves the course's definition of an m-sample: equal marginal inclusion probabilities. It does **not** in general prove a uniform distribution over all m-subsets of the concatenated stream.

For a uniform-subset merge of streams with lengths n_1, n_2, draw $J \sim \text{Hypergeometric}(n_1+n_2, n_1, m)$, then uniformly retain J records from the first reservoir and $m - J$ from the second. Each input reservoir must itself be a uniform subset of size $\min\{m, n_i\}$. The hypergeometric choice matches the number of first-stream records in a uniform merged sample; conditional uniform selection gives the correct records. Merely rounding $mn_i/(n_1 + n_2)$ would not prove exact uniformity.

[Exercise STR-5] Estimate red count

Solution and reasoning.

Let X_i indicate whether red record i is in the sample. If there are R red records, $X_S = \sum_{i=1}^{R} X_i$ and

$$\mathbb{E}\left[\frac{n}{m}X_S\right] = \frac{n}{m}\sum_{i=1}^{R}\frac{m}{n} = R.$$

Independence among inclusion indicators is unnecessary.

[Exercise STR-6] Boyer-Moore plus Sticky Sampling

Solution and reasoning.

Run Sticky with $\phi = 1/2$ and return Boyer-Moore candidate only if it is in Sticky's output. If a majority exists, Boyer-Moore identifies it and Sticky retains it with probability at least $1 - \delta$. If no majority exists, the algorithm returns `null` or an item with frequency at least $(1/2 - \epsilon)n$. It is still not an exact one-pass majority detector.

[Exercise STR-7] Amplify Probabilistic Counting

Solution and reasoning.

One instance has only constant-factor accuracy. Run an odd number L of independent copies and take their median. With $c = 16$ and fully independent hashes within each copy, one-sided failure is at most $1/16$. A bad median needs at least $(L + 1)/2$ bad copies. Chernoff gives probability at most $2^{-(L+1)/2}$ in the course solution. Choosing $(L + 1)/2 \geq \log_2 |U|$ gives one-sided failure at most $1/|U|$.

With pairwise independence within each copy, use the proved one-sided bound $1/8$: union bound over bad-row subsets gives $2^L(1/8)^{L/2} = 2^{-L/2}$. Thus an odd $L \geq 2\log_2 |U|$ also suffices for either tail. For both tails together at most $1/|U|$, replace $|U|$ by $2|U|$.

Independence between copies is essential. The median works because both low and high outliers are possible; a minimum would only address overestimation.

[Exercise STR-8] Estimate the second frequency moment

Solution. Use $\widetilde{F}_{2,j} = \sum_b C[j, b]^2$. Expand using unordered pairs; independent random signs cancel every cross term in expectation. The complete expansion, variance calculation, width, row count and median proof appear in the theorem **Unbiased second-moment row estimator** above. Use ϵF_2 as the error bound.

[Exercise STR-9] Weighted second moment

Solution and reasoning.

Each measurement is (u, z) and aggregate signal is $f_u = \sum z$. Apply signed update $zg_j(u)$, then use $\widetilde{H}_{2,j} = \sum_b C[j, b]^2$. The same cross-term cancellation gives $\mathbb{E}[\widetilde{H}_{2,j}] = \sum_u f_u^2$. Each row is unbiased; median improves concentration but is not automatically unbiased. If the exercise asks "whether estimator is biased," specify which estimator you mean.

[Exercise STR-10] Two-heavy-item Count-Min instance

Solution and reasoning.

Frequencies are $f_a = f_b = n/2 - 5$, with 10 other occurrences, and $w = 2$. If a and b use different cells in a row, all collision noise for a comes from the 10 other occurrences, so

$$R = \frac{10}{n/2 - 5}$$

is a tight relative-error upper bound. Uniform hashing places a and b in different cells with probability $1/2$, so a row meets this bound with probability at least $1/2$. The minimum fails it only if every row fails, giving

$$\Pr\left(\frac{\tilde{f}_a - f_a}{f_a} \le R\right) \ge 1 - 2^{-d}.$$

The official note says equality is approached for large n; "at least" is the robust claim because even a collision between a and b need not be the only way to reason about all special configurations.

[Exercise STR-11] Fold a Bloom filter in half

Solution and reasoning.

Define

$$B[i] = A[i] \,\mathrm{OR}\, A[i + n/2], \qquad 0 \le i < n/2.$$

Query with $h'_j(x) = h_j(x) \bmod (n/2)$. This costs $O(n)$ to construct and preserves no false negatives. Under the exercise's uniformity assumption,

$$\Pr(B[i] = 0) = \left(1 - \frac{2}{n}\right)^{km} \approx e^{-2km/n}.$$

Compression increases the false-positive rate because bits are denser.

[Worked exam question] Conditional Reservoir probability

Solution and reasoning.

Given that x_t was inserted, an old x_i is in the new reservoir iff it was in the old one and was not the uniformly evicted record:

$$\Pr(x_i \in S_t \mid x_t \in S_t) = \frac{m}{t-1}\left(1 - \frac{1}{m}\right) = \frac{m-1}{t-1}.$$

Do not answer m/t: conditioning on inclusion of x_t leaves only $m-1$ slots for the first $t-1$ records.

[Worked exam question] Bloom-filter union

Solution and reasoning.

If A_1, A_2 use the same size and hashes, define $A = A_1 \,\mathrm{OR}\, A_2$. This is precisely a Bloom filter for $S_1 \cup S_2$. If $m = |S_1 \cup S_2|$, then

$$\Pr(A[i] = 0) = \left(1 - \frac{1}{n}\right)^{km} \approx e^{-km/n}.$$

Do not replace m by $|S_1| + |S_2|$ if their intersection was inserted redundantly into both filters; the OR state corresponds to distinct inserted keys, though dependence details may matter for an exact probability calculation.

7. Similarity search

Sources: `8.SimSearch2526-1`, `9SimSearch2526-2`, `EX-SIMSEARCH2526.pdf` and example tests. No matching lecture PDFs for these two note files are present in `Slides/`, so the notes and official exercise sheet are primary local sources for this module.

7.1 Problem definitions - keep quantifiers exact

Let P contain n points in metric space (M, d) and $B_r(q) = \{p \in M : d(p, q) \leq r\}$.

r-Near Neighbor Search (r-NNS). Preprocess P. Given query q and radius r, return any $p \in P \cap B_r(q)$ if this set is nonempty; return `null` if it is empty.

Nearest Neighbor Search (NNS). Given q, return a point minimizing $d(p, q)$ over $p \in P$. It has no radius parameter.

Range Reporting (RR). For $P \subseteq \mathbb{R}^D$, given an axis-aligned rectangle

$$R = [a_1, b_1] \times \cdots \times [a_D, b_D],$$

return **all** points in $P \cap R$. RR asks for all points in a rectangle; r-NNS asks for one point in a metric ball. Their query shapes and output requirements both differ.

(c, r)-Approximate Near Neighbor Search (ANNS), $c > 1$. Preprocess P so that:

- if $P \cap B_r(q) \neq \emptyset$, return some $p \in P$ with $d(p, q) \leq cr$;
- if $P \cap B_r(q) = \emptyset$, either return `null` or a point within distance cr.

The data structure must never return a point farther than cr; verify candidates by actual distance. Approximation relaxes returned distance, not the input promise. ANNS is not the same as returning a multiplicative approximation to exact nearest-neighbor distance.

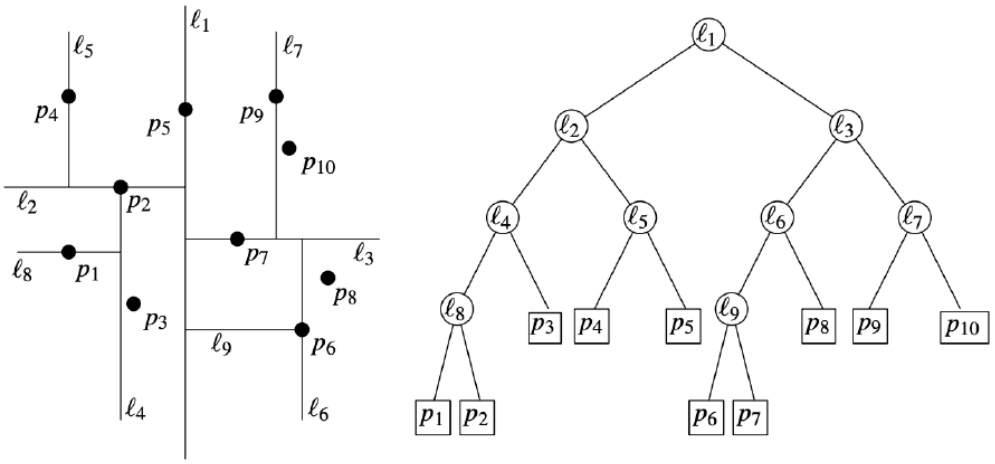
Brute force stores P and scans it. In $\mathbb{R}^D$, space is $O(Dn)$ and query time $O(Dn)$. Construction can be $O(Dn)$ when copying the input; some course tables write $O(n)$ under a convention where storing a point is one operation. State your convention.

7.2 kd-trees

A two-dimensional **kd-tree** is a balanced binary space-partition tree. Root region contains all points. At even depths split by a vertical median line; at odd depths split by a horizontal median line. Each child represents its half-region and about half the points. A leaf stores one point. In D dimensions, cycle through coordinates.

Median splits give logarithmic height, but a rectangular range query can branch into both children. The standard query recursively visits every child region intersecting query rectangle, and reports a leaf exactly when its point lies inside.

kd-tree



17

Figure 10 — Two-dimensional kd-tree: spatial splits and corresponding tree.

For n points in $\mathbb{R}^2$:

$$\text{construction } O(n \log n), \qquad \text{space } O(n), \qquad \text{query } O(\sqrt{n} + k),$$

[Theorem] Two-dimensional kd-tree range reporting
A balanced kd-tree on n planar points has $O(n)$ space, $O(n \log n)$ construction time and $O(\sqrt{n}+k)$ query time, where k is output size.
Proof. There are n leaves and $n-1$ internal nodes, giving linear space. Linear-time median selection at each node gives construction recurrence $T(n) = 2T(n/2) + O(n)$, hence $O(n \log n)$. For range reporting, distinguish boundary-intersecting nodes Q_1 from fully contained subtrees Q_2. A fixed vertical query side crosses one child at vertical splits and at most two at horizontal splits. Over two levels its boundary recurrence is $B(n) \leq 2B(\lceil n/4 \rceil) + O(1)$, giving $O(\sqrt{n})$; horizontal sides are symmetric. Four sides therefore account for $O(\sqrt{n})$ boundary work. Maximal fully contained subtrees are disjoint, have a total of k output leaves, and fewer than $2k$ nodes. Reporting them costs $O(k)$. Together these give $O(\sqrt{n}+k)$. Assume balanced splits with consistent boundary ownership and standard tie handling for equal coordinates (or symbolic perturbation); do not send all equal-coordinate points to one child.

[Theorem] kd-tree range reporting in fixed dimension D
For fixed $D \geq 2$, the course bounds are $O(Dn)$ space, $O(Dn \log n)$ construction and $O(Dn^{1-1/D}+k)$ range-query time. Here outputting a point identifier costs $O(1)$; copying all output coordinates costs $O(Dk)$.
Proof. Cycle through the D coordinates. In D levels a query face branches at most 2^{D-1} ways, while each subtree contains $O(n/2^D)$ points. Thus its boundary recurrence has exponent $\log_{2^D}(2^{D-1}) = 1-1/D$. Sum over the $2D$ faces and charge contained subtrees to output leaves as in the planar proof. Reading/storing coordinates gives the space and construction factors. Constants depend on fixed D. For $D = 1$, use the separate bound $O(\log n + k)$; substituting 1 in the displayed exponent would incorrectly omit search-path cost. As D grows, the exponent approaches 1.

To answer Euclidean r-NNS in $\mathbb{R}^2$, range-report on the axis-aligned square $[q_x - r, q_x + r] \times [q_y - r, q_y + r]$, then test

reported points against the circle. Correctness is exact because the circle lies inside the square and candidates are filtered. Time is $O(\sqrt{n} + k_{square})$, which can be large even when few points lie inside the circle.

7.3 Locality-Sensitive Hashing

A random family $\mathcal{H}$ is (c, r, p_1, p_2)**-locality sensitive** when $p_1 > p_2$ and:

$$d(p,q) \le r \implies \Pr_{h\sim\mathcal{H}}[h(p) = h(q)] \ge p_1,$$

$$d(p,q) > cr \implies \Pr_{h\sim\mathcal{H}}[h(p) = h(q)] \le p_2.$$

The family says nothing mandatory about the grey zone $r < d(p,q) \le cr$. Hash collisions are candidates, not proof of proximity.

Basic one-table structure. Choose random $h \in \mathcal{H}$ and store each p in bucket $T[h(p)]$. For query q, scan bucket $T[h(q)]$, compute actual distances and return the first point within cr, or `null`.

[Theorem] Basic LSH performance for (c, r)-ANNS
Return the first colliding point within distance cr, or `null` after exhausting the bucket. If an r-near point exists, success is at least p_1. If no point is within cr, the answer is always `null`. Expected query time is $O(D(1 + np_2))$, including an $O(D)$ hash evaluation. The course writes $O(Dnp_2)$, suppressing the additive term when $np_2 = \Omega(1)$.
Proof: correctness. Fix a near point p^* with $d(p^*, q) \le r$. With probability at least p_1 it shares the query bucket. Scanning either returns it or an earlier valid point. Every returned point passes the distance test; with an empty cr-ball no point can pass. In the grey zone $r < d(p,q) \le cr$, either a valid point or `null` is allowed when no r-near point exists.
Proof: time. Let X count colliding points with distance greater than cr. Linearity and far-collision control give $\mathbb{E}[X] \le np_2$. Before termination, every rejected candidate is far; at most one accepted candidate is inspected. Hence the number of distance tests is at most $X + 1$, for every bucket order. Grey-zone points terminate the scan when encountered and do not create an extra unbounded cost. Each distance costs $O(D)$. Construction and point storage are $O(Dn)$ under $O(D)$ hash evaluation and ordinary bucket access. An algorithm reporting the whole bucket would require a different output-sensitive bound.

7.4 Bit sampling for Hamming distance

For $x \in \{0,1\}^D$, choose coordinate i uniformly and let $h_i(x) = x_i$. Two vectors differ in exactly $d_H(p,q)$ coordinates, so

$$\Pr(h_i(p) = h_i(q)) = 1 - \frac{d_H(p,q)}{D}.$$

Therefore bit sampling is

$$\left(c, r, 1 - \frac{r}{D}, 1 - \frac{cr}{D}\right)\text{-sensitive},$$

provided thresholds keep probabilities meaningful and the far condition is interpreted with the exercise's strict/non-strict convention. Hamming distance equals L_1 on Boolean vectors.

The LSH exponent is

$$\rho = \frac{\log(1/p_1)}{\log(1/p_2)} = \frac{\log p_1}{\log p_2} \in (0,1).$$

Both log ratios are equal; base is irrelevant. Smaller ρ is better. For bit sampling and small r/D, $\log(1-x) \sim -x$, giving $\rho \sim 1/c$.

[Theorem] Bit sampling for Hamming distance
For binary vectors in $\{0,1\}^D$, choose coordinate i uniformly and let $h_i(x) = x_i$. Then $\Pr(h_i(x) = h_i(y)) = 1 - d_H(x,y)/D$. For $0 < r < cr < D$, this is an $(c, r, 1 - r/D, 1 - cr/D)$-sensitive family.
Proof. Exactly $D - d_H(x,y)$ coordinates agree; divide by D. The probability decreases with distance, giving both LSH inequalities. Independent concatenation of k samples raises the collision probability to its kth power. With $t = r/D$, $-\ln(1-ct) \geq c[-\ln(1-t)]$ (differentiate in t, starting at zero), so $\rho = \ln(1/(1-t))/\ln(1/(1-ct)) \leq 1/c$.

7.5 Random-projection LSH for Euclidean distance

The course gives the family

$$h_{a,b}(p) = \left\lceil \frac{\langle a, p\rangle + b}{w} \right\rceil,$$

where a has independent standard normal coordinates, b is uniform in $[0, w]$, and w is bucket width. Some references use floor rather than ceiling; with continuous b, bucket boundaries differ only by convention.

Random projection turns Euclidean displacement into a one-dimensional normal displacement; the random shift avoids privileged bucket boundaries. Nearby points have higher collision probability. Course notes state $\rho = O(1/c)$ for this family and mention improved Euclidean families with $O(1/c^2)$. Detailed integral derivation of p_1, p_2 is not part of supplied proof material.

7.6 OR and AND amplification

[Theorem] Amplified LSH schema for (c, r)-ANNS
With $k = \lceil \log_{1/p_2} n \rceil$, $\ell = \lceil 2p_1^{-k} \rceil$ and $\rho = \log(1/p_1)/\log(1/p_2)$, success is at least $1/2$ and expected query time is $O(Dn^\rho \log_{1/p_2} n)$ under course assumptions.
Proof and parameter choice.
OR / repetition. Build ℓ independent tables. A fixed near point fails to collide in every table with probability at most $(1-p_1)^\ell$, so success is at least

$$1 - (1-p_1)^\ell.$$

This raises success, but also checks more buckets and admits more far collisions.
AND / concatenation. Define

$$g(x) = (h_1(x), \ldots, h_k(x))$$

with independent hashes. Near collision probability is at least p_1^k; far collision probability is at most p_2^k. This filters far points but also lowers near success.
Choose

$$k = \left\lceil \log_{1/p_2} n \right\rceil.$$

Ignoring rounding, $p_2^k = 1/n$ and expected far collisions per table are at most 1. Moreover,

$$p_1^k = n^{-\rho}.$$

Restore constant near success with

$$\ell = \lceil 2p_1^{-k} \rceil \approx 2n^\rho.$$

Failure in all tables is bounded by

$$(1-p_1^k)^\ell \leq e^{-\ell p_1^k} \leq e^{-2} < 1/2.$$

Thus success is at least $1 - e^{-2} > 1/2$; lecture states the weaker $1/2$.
Course performance bounds are:

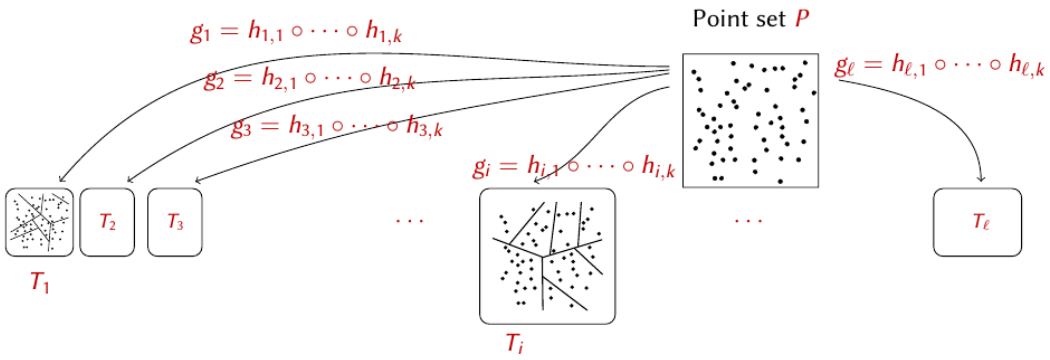
$$\text{construction} = O(Dn^{1+\rho} \log_{1/p_2} n),$$
$$\text{space} = O(Dn + n^{1+\rho} \log_{1/p_2} n),$$
$$\text{expected query} = O(Dn^{\rho} \log_{1/p_2} n).$$

Interpretation: concatenation makes each table selective; repetition restores recall. For a desired failure probability δ, use enough independent repetitions for logarithmic amplification, increasing ℓ by a factor $O(\log(1/\delta))$. Concatenating k bit samples selects k coordinates, with replacement under independent sampling. Collision probabilities become $(1 - d_H/D)^k$. Concatenation changes p_1, p_2 to powers but preserves

$$\frac{\log(1/p_1^k)}{\log(1/p_2^k)} = \rho.$$

These asymptotic forms treat $0 < p_2 < p_1 < 1$ as fixed constants. Rounding gives $p_1 n^{-\rho} \le p_1^k \le n^{-\rho}$ and hence $\ell = O(n^{\rho}/p_1)$. Each table needs k hash evaluations; expected rejected candidates total at most $\ell n p_2^k \le \ell$. Early stopping adds at most one accepted candidate. This proves the displayed expected query cost without a grey-zone assumption.

LSH and (c, r)-ANNS: example



40

Figure 11 — Amplified LSH: concatenated hashes define several independent tables.

7.7 Similarity-search exam checklist

For any answer, state:

1. query promise and legal outputs;
2. data structure construction;
3. bucket or tree regions inspected;
4. exact candidate verification;
5. success probability and which event produces it;
6. expected candidate count and cost per distance computation.

Common mistakes:

- defining NNS when asked for r-NNS;

- returning every RR point when one near neighbor suffices, without stating output cost;
- treating collision as proof that distance is small;
- swapping p_1 and p_2;
- claiming OR reduces false collisions or AND raises success;
- forgetting that kd-tree query cost contains output size k;
- confusing LSH concatenation length k with k-nearest neighbors or k-center.

7.8 Exercises

[Exercise SIM-1] Return one range point

Solution and reasoning.

Store at each node v an arbitrary representative p_v from its subtree. During the range query, follow boundary-intersecting nodes. As soon as some node region lies wholly in R, return p_v immediately. If no such node appears, inspect reached leaves. At most $O(\sqrt{n})$ boundary nodes and one contained node are processed, so query time is $O(\sqrt{n})$, independent of output size.

[Exercise SIM-2] Document search with bit sampling

Solution and reasoning.

Represent document p by Boolean vector V_p over relevant vocabulary $W = \{w_0, \dots, w_{D-1}\}$, where bit i says whether word w_i appears. Use Hamming distance and one bit-sampling hash table.

To obtain success at least $1/2$, require $p_1 = 1 - r/D = 1/2$, hence $r = D/2$. To obtain expected query $O(n)$ instead of $O(Dn)$, require $p_2 = O(1/D)$. Choose constant $a > 1$ and

$$c = 2\left(1 - \frac{a}{D}\right), \qquad cr = D - a,$$

giving $p_2 = a/D$. This requires $D > 2a$ so $c > 1$. It is a formal parameter exercise; whether Hamming distance is the best semantic measure for shared words is a separate modeling issue.

[Exercise SIM-3] Far Hamming search

Solution and reasoning.

Opposite sampled bits occur precisely on coordinates where vectors differ:

$$\Pr[h(p) = \text{not}(h(q))] = \frac{d_H(p,q)}{D}.$$

Scan opposite bucket $T[\text{not}(h(q))]$ and verify distances, returning first p with $d_H(p,q) \geq r$. If an r-far point exists, it lands there with probability at least r/D. This reverses the usual LSH search: far vectors are more likely to collide with the **complement** of the query hash.

8. Current homeworks and older homework questions

Sources: `Homeworks/BDC_HW1.md`, `Homeworks/Results HW1.md`, `PROJECT_DESCRIPTIONS_EXAM` and two transcribed voice notes in `Slides/Exercises/Audio`. The audio explicitly recalls questions about `foreachRDD`, m-samples, Reservoir Sampling, LSH amplification and formulating Fair-FFT.

8.1 Current HW1 - quota-constrained fair k-center

Each point $p \in U \subseteq \mathbb{R}^D$ has label $g_p \in \{A, B\}$. Given $k_A \leq |U_A|$ and $k_B \leq |U_B|$, choose $S \subseteq U$ with exactly k_A A-centers and k_B B-centers to minimize

$$\Phi_{fair\text{-}kcenter}(U, S) = \max_{x \in U} \min_{s \in S} \|x - s\|_2.$$

Fairness constrains selected-center counts. It does not require points to be assigned to centers of their own group and it does not balance cluster sizes.

Fair-FFT. Maintain selected set S, used quotas and cached distance $D[x] = d(x, S)$. Pick an initial point from a group with positive quota. At every subsequent iteration choose the eligible point maximizing $D[x]$: a point is eligible only when its group's quota is not exhausted. After selecting c, update

$$D[x] \leftarrow \min\{D[x], d(x, c)\}.$$

Continue until $k_A + k_B$ distinct centers have been selected. Complexity is $O(NkD)$ time and $O(N + k)$ auxiliary storage. Preconditions on group sizes must be checked. Input order can affect the arbitrary first center and tie-breaking.

This greedy quota modification is the homework algorithm. Do not automatically claim the standard FFT 2-approximation proof: eligibility restrictions break its unrestricted farthest-point choice, and no approximation theorem for this exact heuristic is stated in supplied assignment.

MR-Fair-FFT. Use L Spark partitions. Within each partition, `mapPartitions` runs Fair-FFT with local oversampling quotas, for example $k'_A = 2k_A, k'_B = 2k_B$, and emits at most $k'_A + k'_B = 2k$ local representatives. Collect their union T, at most $2kL$ points, then run Fair-FFT on T with final quotas k_A, k_B.

Main tradeoff:

- larger L means smaller partitions and potentially more parallelism;
- coreset size, task overhead, communication and driver work grow with L;
- larger local quotas can improve geometric coverage, but enlarge T;
- each partition converted to a list must fit executor memory;
- `collect()` is safe only while $|T| = O(k'L)$ fits driver memory.

The final objective must be computed over the original distributed U, not only over T. If benchmarking only MR-Fair-FFT, materialize inputs first and keep loading and the final objective pass outside the timed region.

Likely four-point response: give formal objective and quota constraints; describe eligible farthest selection; describe partition-local oversampling plus final run; mention $|T| \leq (k'_A + k'_B)L$, driver limitation and L tradeoff.

8.2 Current HW2 - frequent items with Spark Streaming

The project processes first n valid integers from a socket stream and compares Sticky Sampling with Count-Min Sketch.

Spark Streaming exposes a **DStream**, a sequence of micro-batch RDDs. `foreachRDD` registers a function applied to each batch RDD. In the project, records are parsed and delivered through `toLocalIterator()` to the driver-side updates. Small algorithm state persists between callbacks. The process stops exactly after n valid records; malformed input is skipped.

Sticky branch: use

$$r = \frac{\ln(1/(\delta\phi))}{\epsilon}, \qquad p = \min\{r/n, 1\};$$

sample previously untracked values with probability p, increment tracked counters, and return counters at least $(\phi - \epsilon)n$.

Count-Min branch: increment $C[j, h_j(x)]$ for every row, estimate by row minimum, and add an observed item to candidate output once its estimate reaches ϕn. Since estimates only increase, an item crossing the threshold remains a candidate.

Aspect	Sticky Sampling	Count-Min
Error source	Missed prefix before sampling	Hash collisions
False negatives	Controlled probability	None for insertion-only updates at same threshold
False positives	Only grey-zone items under successful event	Collision-inflated rare items possible
Main parameters	smaller ϵ, δ increase r and memory	larger w cuts collisions; larger d raises confidence
Update	$O(1)$ expected	$O(d)$
Stored state	Random hash table	Fixed matrix plus candidate set

`trueFrequencies` is experimental ground truth and can grow with number of distinct values. It must not be counted as part of claimed memory benefit. `toLocalIterator()` avoids one full-batch list but still makes approximate updates sequential on driver. At-least-once processing can duplicate a batch after failure unless checkpointing/deduplication handles it.

Expected experiment trends: increasing ϵ shrinks Sticky sample but worsens accuracy; increasing w reduces Count-Min false positives; increasing d improves reliability and costs more time and memory. Random trials need repetitions even on same input.

8.3 Older fair k-means objective - do not confuse it with current HW1

Example 5 asks about an older **group-fair k-means** objective. A standard formulation used by that question is

$$\Phi(A, B, C) = \max \left\{ \frac{1}{|A|} \sum_{a \in A} d(a, C)^2, \frac{1}{|B|} \sum_{b \in B} d(b, C)^2 \right\}.$$

It balances average squared cost between demographic groups. Current fair k-center instead imposes quotas on which input points become centers and minimizes one maximum service radius.

For $n - 1$ A-points at 0 and one B-point at 1 with one arbitrary centroid c:

- standard k-means centroid is mean $c = 1/n \to 0$;
- fair group costs are c^2 and $(1 - c)^2$;
- minimizing their maximum equalizes them, giving $c = 1/2$.

The plain-text recollection labels this as fair k-center, while actual `ExampleWT-5.pdf` says fair k-means and shows this centroid setup. Trust the original exam PDF for that old question.

8.4 How to answer a homework question

Use this order:

1. formal objective and feasibility constraints;
2. algorithm data flow, including which data remain distributed and which reach driver;
3. purpose of each parameter;
4. complexity and scaling bottleneck;
5. guarantee and source of error;
6. one implementation limitation or experimental trend.

Avoid reciting code line by line. Explain why implementation matches the big-data model.

9. Exercises: answer keys for the five example written tests

These keys assume wording in `ExampleWT-1.pdf` through `ExampleWT-5.pdf`. Long algorithms refer to full derivations above, but each key contains the exact core expected in an answer.

[Practice exam] Example Written Test 1

[Question 1.1] cluster capacity

One reducer must fit one worker's RAM, so $M_L \leq 8$ GB. Total HDFS disk is $10 \cdot 128 = 1280$ GB, so idealized $M_A \leq 1280$ GB. Mention that real HDFS replication and Spark overhead reduce usable capacity if considered.

[Question 1.2] MR k-means coreset

Round 1 runs a sequential unweighted k-means method on each partition, obtaining local representatives T_i. For $q \in T_i$, weight

$$w(q) = |\{x \in P_i : q \text{ is closest representative to } x\}|.$$

Final weighted solver runs on $T = \bigcup_i T_i$. Weight is represented multiplicity, not geometric distance or original input weight unless the weighted-input variant says so.

[Question 1.3] conditional Reservoir probability

Given x_t was inserted, one reservoir slot is occupied by x_t and $m-1$ uniformly represented old slots remain:

$$\Pr(x_i \in S_t \mid x_t \in S_t) = \frac{m-1}{t-1}.$$

Derive as $[m/(t-1)](1-1/m)$.

[Question 1.4] (c, r)-ANNS

If some point lies within r, return a point within cr; otherwise `null` or a point within cr is legal. Exact r-NNS must return a point inside r when one exists. In both, do not output beyond the permitted radius.

[Question 2.1] biased slot machines

Three rounds: locally count (s, o) after balanced ID partitioning; globally sum each (s, o) and retain totals at least $N/50$; group retained outcomes by s and emit one $(s, null)$. Final reducer gets at most 50 outcomes because every retained pair consumes at least $N/50$ records. $M_L = O(\sqrt{N})$, $M_A = O(N)$.

[Question 2.2] sensor second moment

Use Count Sketch with signed weighted updates: for measurement (u, w_i) add $w_i g_j(u)$ to $C[j, h_j(u)]$. Row estimate $\sum_b C[j, b]^2$ is unbiased for $\sum_u f_u^2$ because cross terms cancel in expectation. Use independent rows and median for concentration; distinguish row unbiasedness from median.

[Practice exam] Example Written Test 2

[Question 1.1] `mapPartitions`

Function receives iterator for one RDD partition and emits an iterator of zero or more results. It can build a local summary once per partition, such as counts or centers. Its local state must fit memory; it is not called once per record.

[Question 1.2] weighted k-means++

For current centers S,

$$\Pr(y \text{ selected next}) = \frac{w(y)d(y, S)^2}{\sum_{x \in P \setminus S} w(x)d(x, S)^2}.$$

State weighted first-center rule separately if asked.

[Question 1.3] Sticky memory

Let I_t indicate that arrival t creates a new table entry. $\Pr(I_t = 1) \leq r/n$, hence $\mathbb{E}|S| \leq r$ by linearity. Therefore expected memory is

$$O(r) = O\left(\frac{\ln(1/(\delta\phi))}{\epsilon}\right).$$

[Question 1.4] RR and kd-tree

RR reports all points in an axis-aligned rectangle; NNS asks for closest point, with different query and output. In $\mathbb{R}^2$, kd-tree uses $O(n)$ space and answers RR in $O(\sqrt{n}+k)$ time.

[Question 2.1] interval k-center

Choose one input point c_i from each interval $[(i-1)/k, i/k]$. Every $x \in P$ lies in one such interval and is within its length $1/k$ of that interval's chosen point. Thus this feasible k-center set has radius at most $1/k$, so

$$\mathrm{OPT}_{kcenter}(P,k) \leq 1/k.$$

Endpoints shared by adjacent intervals cause no problem; assign them consistently.

If interval endpoints cause the same point to be chosen twice, keep distinct chosen points and add unused input points until there are k centers, assuming $|P| \geq k$. Empty intervals need no representative. Adding centers cannot increase the covering radius.

[Question 2.2] net sales

Use update weight $z = +1$ for purchase, -1 for return in Count Sketch. Estimate each row by $g_j(p)C[j, h_j(p)]$ and take median. Each row is unbiased. If every transaction concerns p, there are no other-key collisions; every row equals exact net sales, even though positive and negative records cancel.

[Practice exam] Example Written Test 3

[Question 1.1] MR goals

Target constant rounds, sublinear local space and linear aggregate space. A one-reducer sequential simulation has $M_L = \Theta(N)$, so one worker must store the entire dataset and parallelism is lost.

[Question 1.2] old outlier homework

This concerns a previous year's k-center-with-outliers homework, not current fair k-center. Expected response must follow that assignment's Round 2: merge local summaries/candidates and run final selection. Increasing L shrinks each local partition but grows number of candidates sent to Round 2, increasing final input and driver or reducer work. Do not invent exact formulas without old code parameters.

[Question 1.3] FFT proof

Add farthest point q to k selected centers. Every pair among these $k+1$ points is at distance at least final radius. Two share an optimal cluster, so their distance is at most 2OPT. Therefore FFT radius is at most 2OPT.

[Question 1.4] probabilistic counting

Hash each distinct key, compute trailing-zero count, maintain maximum R, and return 2^R. A hash reaches at least j zeros with probability 2^{-j}, so maximum lies near $\log_2 F_0$.

[Question 2.1] at most K sensor records

Partition by distinct ID. Within each partition retain at most K records per sensor and emit them by sensor. Global reducer retains at most K. With $L = \sqrt{N}$, $M_L = O(\max\{\sqrt{N}, K\sqrt{N}\})$ and $M_A = O(N)$. Constant K gives $O(\sqrt{N})$; $K = \log_2 N$ gives $O(\sqrt{N}\log N) = o(N)$.

[Question 2.2] Bloom union

Set $A = A_1 \,\mathrm{OR}\, A_2$ and query with same hashes. No union member can be a false negative. If $m = |S_1 \cup S_2|$, $\Pr(A[i] = 0) = (1-1/n)^{km} \approx e^{-km/n}$.

[Practice exam] Example Written Test 4

[Question 1.1] cluster requirements

Every local map/reduce invocation and its records must fit available worker memory at least M_L; distributed storage must hold aggregate data at least M_A, with enough workers/tasks to schedule the round and any replication overhead.

[Question 1.2] MR-FFT

Round 1 partitions into L parts of size N/L and runs FFT to emit k centers each. Round 2 runs FFT on at most kL representatives. Therefore $M_L = O(\max\{N/L, kL\})$ and is minimized at $L = \sqrt{N/k}$, giving $O(\sqrt{Nk})$.

[Question 1.3] one-table LSH

Hash query, scan its bucket and verify actual distances, returning first point within cr. Near point collides with probability at least p_1. Expected far collisions are at most np_2 by indicators. At most one valid candidate is checked, so expected query time is $O(D(1 + np_2))$, or $O(Dnp_2)$ when $np_2 = \Omega(1)$.

[Question 1.4] streaming

Metrics: memory, passes, update time and query time, plus accuracy. A Spark DStream is a sequence of micro-batch RDDs. `foreachRDD` handles each batch; iterate its records and apply ordinary item-at-a-time updates while retaining compact state across batches.

[Question 2.1] furthest center by average distance

Partition input. For every local cluster i, accumulate a vector whose component j is sum of $d(x, c_j)$ and retain cluster count. Round 2 merges vectors and counts by i, divides by total count and returns maximizing j. Because denominator is common across candidates for fixed i, maximizing merged sums suffices. With constant k and $L = \sqrt{N}$, local space is $O(\sqrt{N})$ and aggregate $O(N)$.

[Question 2.2] colored weighted frequency

Count-Sketch update is $z_i g_j(u_i)$, with $z_i = 1/2$ for red and $1/3$ for blue. Row query $g_j(u)C[j, h_j(u)]$ is unbiased for $f_{u,red}/2 + f_{u,blue}/3$; median gives final robust estimate. A collision-free row for u is exact because both colors belong to desired signal.

[Practice exam] Example Written Test 5

[Question 1.1] lazy timing

Transformations create lineage and execute only after an action. Timing only transformation construction measures plan creation, not distributed work. Trigger an action, account for persistence/materialization and state which phases timing includes.

[Question 1.2] diameter

If every $x \in P$ lies within R of $T \subseteq P$, then

$$\Delta_T \leq \Delta \leq \Delta_T + 2R.$$

For diameter pair x, y, choose proxies t_x, t_y. Triangle inequality gives $d(x, y) \leq R + d(t_x, t_y) + R$.

[Question 1.3] Sticky guarantees

With probability at least $1 - \delta$, all items with $f_x \geq \phi n$ are returned; no item with $f_x < (\phi - \epsilon)n$ is ever returned. Grey-zone items may be present. Separate expected $O(r)$ memory from this correctness event.

[Question 1.4] old fair k-means

Use group-normalized max objective from Section 8.3. Standard one-center k-means chooses $1/n \to 0$; fair objective chooses $1/2$. This question is not current quota-constrained homework.

[Question 2.1] dense grid rows

Three rounds: deduplicate cells within balanced ID partitions; deduplicate globally by cell and emit one column per row; count distinct columns by row and retain counts above $t/2$. With $t = O(\sqrt{N})$, $M_L = O(\sqrt{N})$ and $M_A = O(N)$.

[Question 2.2] far Hamming search

Opposite sampled-bit probability is $d_H(p, q)/D$. Scan bucket $T[\text{not}(h(q))]$, verify actual distance and return an r-far vector. If one exists, success probability is at least r/D.

10. Last revision - formulas, proof scripts and common mistakes

10.1 One-page formula sheet

Topic	Formula or guarantee
Random L-partition load	$\mathbb{E}[X_j] = N/L$; Chernoff per bin, then union bound
Two-level MR aggregation	$M_L = O(\max\{N/L, L\})$; choose $L = \sqrt{N}$
MR with k outputs per partition	$M_L = O(\max\{N/L, kL\})$; choose $L = \sqrt{N/k}$
Multi-level aggregation	local $O(M)$, rounds $\lceil \log_M N \rceil$, aggregate $O(N)$
FFT	$\Phi(P, S) \leq 2\mathrm{OPT}_{kcenter}$
MR-FFT	$M_L = O(\sqrt{Nk})$, $M_A = O(N)$, approximation factor 4
Direct coreset cover	$d(x, T) \leq R \Rightarrow \Delta(T) \leq \Delta(P) \leq \Delta(T) + 2R$
External-center representative	external radius $R \Rightarrow \Delta(P) \leq \Delta(T) + 4R$
Reservoir Sampling	$\Pr(x_i \in S_t) = m/t$
Reservoir conditional	$\Pr(x_i \in S_t \mid x_t \in S_t) = (m-1)/(t-1)$
Sticky Sampling	$r = \lceil \ln(1/(\delta\phi))/\epsilon \rceil$; expected memory $O(r)$
Sticky output	all $f_x \geq \phi n$ with joint probability $1 - \delta$; none below $(\phi - \epsilon)n$
Probabilistic Counting	$R = \max \mathrm{tr}(h(x))$, $\widetilde{F}_0 = 2^R$
Count-Min	$w = \lceil 2/\epsilon \rceil$, $d = \lceil \log_2(1/\delta) \rceil$, $0 \leq \tilde{f}_u - f_u \leq \epsilon n$
Count Sketch	$w = \Theta(1/\epsilon^2)$, $d = \Theta(\log(1/\delta))$, error $\epsilon\sqrt{F_2}$
Second moment with Count Sketch	4-wise signs; $w = \Theta(1/\epsilon^2)$; error ϵF_2
Bloom filter	$p_0 = (1 - 1/n)^{km} \approx e^{-km/n}$; $p_{FP} \approx (1 - e^{-km/n})^k$
Optimal Bloom hashes	$k = (n/m) \ln 2$
kd-tree in $\mathbb{R}^2$	space $O(n)$; RR query $O(\sqrt{n} + k)$
Basic LSH	success $\geq p_1$; expected query $O(D(1 + np_2))$
Bit sampling	collision $1 - d_H/D$; opposite-bit event d_H/D
LSH exponent	$\rho = \log(1/p_1)/\log(1/p_2)$
Amplified LSH	$k = \log_{1/p_2} n$, $\ell = 2n^\rho$; constant success

All logarithmic row/table counts need ceiling in actual data structures. When a theorem writes $d = \log_2(1/\delta)$, use $d = \lceil \log_2(1/\delta) \rceil$ if an integer is required.

10.2 Ten proof scripts to reproduce from memory

1. Random partition load. Define Bernoulli indicators for one partition; compute mean; apply Chernoff; union bound over all partitions; conclude maximum load with stated probability.

2. FFT factor 2. Add farthest point to selected centers; prove all $k + 1$ points are separated by final radius; pigeonhole into k optimal clusters; triangle inequality gives distance at most 2OPT.

3. MR-FFT factor 4. Apply separation argument locally against global optimum to obtain 2OPT proxy distance; apply it on coreset for another 2OPT; join with triangle inequality.

4. Diameter coreset. Take true diameter endpoints; choose their representatives; follow endpoint -> proxy -> proxy -> endpoint; add two proxy radii.

5. Reservoir Sampling. Induct on time. New record included with m/t; old record was included with $m/(t-1)$ and survives with $1 - 1/t$.

6. Sticky Sampling. Lower-bound counter eliminates deep false positives; missing a frequent item implies missing its first ϵn occurrences; bound with $e^{-\epsilon r}$; union bound over at most $1/\phi$ frequent items. Entry indicators give expected memory r.

7. Count-Min. Express one row's error as nonnegative colliding mass; expectation at most n/w; Markov makes one row bad with probability at most $1/2$; minimum bad only when every independent row is bad.

8. Count Sketch. Express row estimate as truth plus signed collisions; each collision term has mean zero; variance at most F_2/w; Chebyshev yields constant success; median amplifies.

9. Bloom filter. One insertion-hash misses fixed cell with probability $1 - 1/n$; all km miss with power km; complement gives a bit set; all query bits set gives approximate FP. No-false-negative proof follows directly from monotone bit setting.

10. LSH. A near point collides with probability p_1; far-collision indicators have expectation at most p_2 each; sum gives np_2; multiply by $O(D)$ distance cost. For amplification, AND powers collision probabilities and OR complements all-table failure.

10.3 How to invent a MapReduce solution under pressure

1. Identify logical key that defines independent final answers: item, sensor, cell, row, cluster.
2. Ask whether one logical key can occur N times. If yes, direct grouping violates local space.
3. First partition by distinct ID or independent random choice, never by skewed logical key.
4. Inside each balanced partition, compute smallest mergeable summary per logical key.
5. Regroup summaries by logical key and merge them.
6. If the final number of logical outputs can itself be N, aggregate their count or maximum through another balanced level.
7. Prove why each reducer receives at most N/L, L, kL, KL or another explicit bound.
8. Count total summary records. Charge each emitted local key to at least one input record.

For distinctness, use two deduplication layers. For averages, carry sum and count. For top-K, truncate locally and globally. For extrema, keep one candidate per group. For a vector of candidate costs, merge vectors componentwise.

10.4 How to write a short theory answer

Use four sentences or compact equivalents:

1. exact definition;
2. algorithm or formula;
3. guarantee;
4. cost or one-line justification.

Example for Count-Min:

> Count-Min stores a $d \times w$ counter array with one hash per row. Arrival x increments $C[j, h_j(x)]$ in every row, and query returns their minimum. In insertion-only streams it never underestimates; with $w = 2/\epsilon$ and $d = \lceil \log_2(1/\delta) \rceil$, fixed-item error is at most ϵn with probability at least $1 - \delta$. It uses $O(dw)$ counters and $O(d)$ update and query time.

10.5 Frequent traps

- Writing $M_A = O(N)$ without counting intermediate pairs and their payload sizes.
- Ignoring mapper output in M_L, especially during replication.
- Using expected partition size as a high-probability maximum without Chernoff and union bound.
- Hashing a repeated logical key when random assignment must be independent per record.
- Sending all remaining values to one final reducer.
- Forgetting parameters: K, k, D, L, d, w.
- Averaging local averages instead of merging sums and counts.
- Counting repeated events when problem asks for distinct cells or values.
- Claiming FFT's theorem for quota-constrained Fair-FFT without a supplied proof.
- Confusing approximation ratio with success probability.
- Saying Reservoir's expected one copy means guaranteed presence.
- Calling Sticky counters unbiased; they are lower bounds after sampling begins.
- Calling final Count-Sketch median unbiased; theorem proves row estimators unbiased.
- Using Count-Min for signed updates while retaining no-underestimate claim.
- Forgetting exact-distance verification after LSH bucket lookup.
- Confusing OR and AND amplification.
- Reusing old fair k-means objective for current fair k-center homework.

10.6 Final self-test

Without notes, answer each in 10-12 minutes:

1. Design a 3-round exact distinct-cell counter with $M_L = O(\sqrt{N})$.
2. Prove FFT factor 2 and MR-FFT factor 4.
3. Explain three different diameter bounds and when each uses $2R$ or $4R$.
4. Derive Sticky failure probability and expected memory.
5. Derive Count-Min accuracy from expectation through Markov and row independence.

6. Derive Count-Sketch row unbiasedness and variance.
7. Prove Bloom no false negatives and derive its false-positive approximation.
8. Define RR, r-NNS and (c, r)-ANNS without mixing their outputs.
9. Derive bit-sampling collision and amplified LSH parameters.
10. Explain both current homeworks, parameters, bottlenecks and experiment trends.

Then complete two entire example tests under 150-minute conditions. Grade definitions strictly: a plausible explanation with incorrect quantifiers is not a correct theory answer.

11. Sources, coverage and corrections

Coverage

This guide covers:

- all nine main lecture-note files in `Notes/`;
- handwritten proof PDF in `Slides/Theory/BDC_proofs.pdf`;
- all **5 MapReduce**, **13 clustering/coreset**, **11 streaming** and **3 similarity-search** official exercise-sheet problems, totaling 32;
- all six questions in each of the five example written tests;
- supplied old exam questions and available Part 2 solutions;
- current HW1 assignment, available HW1 results, current HW2 summary and two voice notes.

Original PDFs remain authoritative when OCR text or derivative notes disagree. Some slide PDFs contain image-only pages; they were visually checked alongside extracted text where relevant.

Theorem and proof coverage index

Every theorem/proof topic in the nine lecture notes, `BDC_proofs.md`, `TheoremsDefinitionsProofs.md` and the 24 handwritten pages is mapped below. Duplicate source versions are consolidated into one explanation. Exercise proofs remain with their solutions; formal theorem proofs are inside the corresponding named callout.

Source topic	Location in this guide	Proof coverage
MTBF; one-round Word Count; deterministic and random Class Count	Chapters 3–4	Geometric waiting time; mapper/reducer bounds; balanced loads
Probability tools	Chapter 2	Linearity, union, Markov, Chebyshev, both Chernoff tails, median
FFT; local coreset lemma; MR-FFT space and approximation	Chapter 5	Separation, pigeonhole, balancing, proxy paths
Sampling counterexample; diameter lemmas	Chapter 5	Rare isolated point; one-point and representative bounds
Diversity proxy lemma and coreset theorem	Chapter 5	Injection, pair loss, relation to k-center, exact approximation factor
Weighted mean; MR-kmeans theorem	Chapter 5	Squared-distance identity; proxy error; center-domain assumptions
Boyer-Moore; Reservoir; Sticky Sampling	Chapter 6	Cancellation invariant; induction; miss probability and expected memory
Probabilistic Counting and amplification	Chapters 2 and 6	Both tails, integer thresholds, independence distinctions, median
Count-Min; Count Sketch; second moment	Chapter 6	Bias, expectation, variance, confidence amplification
Bloom filters; fingerprints; universal hashing	Chapter 6	No false negatives, approximate error and optimum, collision proof
kd-trees in 2 and D dimensions	Chapter 7	Construction, boundary recurrence, output charging
Basic LSH; Hamming bit sampling; OR/AND and general schema	Chapter 7	Correctness, early stopping, collisions, parameter and cost analysis

Source topic	Location in this guide	Proof coverage
Supplementary proof exercises in consolidated notes	Exercises in Chapters 5–6	CTCL-4/5/9, STR-4/5 and full solutions

Handwritten PDF page map (24 pages). Pages 1–2: MTBF, Word Count and Class Count; 3–5: random partitioning and Chernoff bounds; 6–7: FFT; 8–10: local coresets and MR-FFT; 11: sampling counterexample; 12: diameter; 13–14: diversity; 15–16: Boyer-Moore; 17: Reservoir; 18: Sticky; 19: probabilistic counting; 20: Count-Min; 21–22: Count Sketch; 23: Bloom filters; 24: basic LSH. These topics all have corresponding explanations above.

Statements without a supplied original proof. The notes state the k-means++ expected approximation, the $(2 - 2/k)$ sequential diversity approximation, and asymptotic performance of Euclidean projection LSH without developing their original proofs. This guide preserves those statements and marks that scope. It includes the supplied MR-kmeans proof and the Markov/repetition derivation for k-means++ success. The lower FM tail, full sketch variance and kd-tree boundary recurrence are additional derivations provided here to fill explanatory gaps; their assumptions are stated explicitly. No missing source proof is described as if it had been present in the handwritten PDF.

Corrections and cautions found during review

1. **Exam-pattern overstatement:** one existing study plan says every example contains a long MapReduce exercise. Example 2 does not. Four of five do.
2. **Current versus old fairness:** actual Example 5 asks group-fair k-means; current homework is quota-constrained fair k-center. A plain-text recollection conflates them.
3. L_1 **terminology:** clustering sheet calls L_1 "Euclidean"; standard name is Manhattan. Euclidean is L_2.
4. **Count-Sketch medians:** individual row estimates are unbiased. Median inherits accuracy, not automatic unbiasedness.
5. **Second-moment bound:** course Slide 47 prints error $\epsilon\sqrt{F_2}$ for estimating F_2, which is dimensionally inconsistent. Standard derivation gives relative ϵF_2 error for this width. State the corrected theorem and identify the source typo.
6. **Diameter constants:** direct R-cover gives additive $2R$; replacing external cluster centers by arbitrary input representatives gives additive $4R$.
7. **Angular distance domain:** it becomes a true metric only after normalizing or identifying positive scalar multiples; zero vectors require separate handling.
8. **Basic LSH query bound:** early stopping is already part of the algorithm. At most $X + 1$ candidates are checked; no random bucket order or extra grey-zone assumption is needed.
9. **PAM runtime phrasing:** $O(Nk)$ is number of candidate swaps in a naive iteration, not necessarily total time to evaluate them.
10. **Sticky ceiling:** mathematically correct rate is $\lceil \ln(1/(\delta\phi))/\epsilon \rceil$ and sampling probability is capped at 1.

Primary navigation links

- MapReduce and Spark: `1.MapReduce2526`, `2.Spark2526`, `3.WordCountSpark`
- Coresets: `4.Coreset2526-1`, `5.Coreset2526-2`
- Streaming: `6. Streaming2526-1`, `7.Streaming2526-2`
- Similarity Search: `8.SimSearch2526-1`, `9SimSearch2526-2`
- Proof references: `BDC_proofs`, `TheoremsDefinitionsProofs`
- Practice bank and projects: `ExamStyleQuestions`, `PROJECT_DESCRIPTIONS_EXAM`

External verification was limited to primary sources where local material had an ambiguity: Apache's official RDD guide confirms lazy transformations and action-triggered execution, while original Count Sketch literature supports random-sign cancellation. Exam preparation should follow professor terminology and explicit exercise assumptions.